

Beyond Pure Sampling: Hybrid Optimization Mechanisms for Non-Convex Model Predictive Control

Journal Title
XX(X):1–28
©The Author(s) 2016
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Yuichiro Aoyama^{1,2}, Minchan Jung^{1,3*}, Akash Ratheesh^{1*}, and Evangelos A. Theodorou¹

Abstract

This paper investigates the optimization mechanisms of non-convex Model Predictive Control (MPC) using the Maximum Entropy Differential Dynamic Programming (ME-DDP) framework. Navigating non-convex cost landscapes induced by nonlinear dynamics, multiple obstacles, etc. remains a fundamental challenge in robotics, where gradient-based methods frequently converge to suboptimal local minima. We demonstrate a dual-step optimization mechanism designed to overcome these traps. (1) an initial phase of using DDP to exploit the gradient of the cost landscape, followed by (2) disruption of the optimization via sampling from policies characterized by the inverse Hessian of the action-value function. We provide a rigorous analysis of this sampling mechanism of three ME-DDP variants: Unimodal Gaussian ME-DDP, Multimodal Gaussian ME-DDP, and Stein Variational DDP. Furthermore, with navigation tasks of four robotic systems under cluttered environments, we conduct extensive benchmarking of three variants of the ME-DDP, against deterministic DDP, and one of the most successful sampling-based schemes, Model Predictive Path Integral (MPPI) control with three policy parameterizations and update laws that correspond to those of ME-DDPs. The results show that in low-dimensional systems where the cost landscapes are relatively simple and local information is sufficiently representative, our framework consistently outperforms MPPIs. In high-dimensional systems, MPPI can occasionally discover aggressive maneuvers that enable it to steer the systems faster than DDP-based methods, whereas our method maintains a higher, more stable success rate. Finally, we validate the practical efficacy of the framework through hardware experiments with a quadrotor navigating a dense, non-convex obstacle field, confirming the robustness of the proposed framework for real-world deployment.

Keywords

Optimal Control, Stochastic Optimal Control, Differential Dynamic Programming

1 Introduction

Trajectory Optimization (TO) is a fundamental technique for enabling robotic and autonomous systems to navigate safely and efficiently (Kuindersma et al. 2016; Tassa et al. 2014). From an optimization perspective, TO in robotics is characterized by nonlinear dynamics, state and actuation constraints, and complex environmental geometries, such as cluttered environments with multiple obstacles. The interplay of these factors induces a non-convex cost landscape (Schulman et al. 2014; Deits and Tedrake 2015). A canonical example is found in navigation tasks in cluttered environments, where the presence of multiple local minima, i.e., different topological paths around obstacles, creates multiple solutions (Osa 2020).

To make trajectory optimization computationally tractable, researchers have traditionally relied on local approximations of this landscape. Consequently, gradient-based families of algorithms, such as Differential Dynamic Programming (DDP) (Jacobson and Mayne 1970) and Sequential Quadratic Programming (SQP) for dynamical systems (Wilson 1963; Gill et al. 2000, 2002), have been widely used.

DDP and its variants, such as the iterative Pontryagin Maximum Principle (iPMP) (Kaufman 1964), utilize a forward-backward sweep architecture to propagate value functions or costates, arising from quadratic or

linear approximation of cost and dynamics. Although this structure offers scalability and speed, it relies on local information (gradient) and approximations around nominal trajectories. Similarly, SQP addresses TO by solving quadratic subproblems derived from quadratic approximation of the cost and linearized dynamics.

Because both families rely strictly on local first- and second-order information, they are both vulnerable to being captured by sub-optimal solutions. In the context of robotic tasks, this may correspond to situations where the robot gets stuck at a dead end or finds an inefficient path even if it reaches the target. In addition, the local nature makes the algorithm sensitive to the initial condition and incremental (Kalakrishnan et al. 2011; Zucker et al. 2013).

¹ School of Aerospace Engineering, Georgia Institute of Technology, Atlanta, GA, USA

² Development Division, Komatsu Ltd., Tokyo, Japan

³ Department of Electrical and Computer Engineering, Inha University, Incheon, Republic of Korea

* Equally contributed.

Corresponding author:

Yuichiro Aoyama, School of Aerospace Engineering, Georgia Institute of Technology, 270 Ferst Drive, Atlanta, GA, USA

Email: yaoyama3@gatech.edu

On the opposite side of gradient-based approach, sampling-based Stochastic Optimization (SO) methods overcome the limitations of locality. These algorithms, typically deployed in a Model Predictive Control (MPC) fashion, have been successfully used across diverse robotic systems. The most representative algorithms in this approach are Cross-Entropy (CE) and Model Predictive Path Integral control (MPPI) and its variations (Williams et al. 2016, 2017, 2018a,b; Gandhi et al. 2021; Wang et al. 2021). The algorithms sample dynamics forward and with perturbed control and refine their policy by weighting the trajectories based on the cost associated with the samples.

Because these algorithms do not require derivatives of the cost and dynamics, they are easily applied to non-smooth systems. Furthermore, the stochastic sampling process enables broader exploration, allowing algorithms to explore multiple local solutions that gradient-based methods cannot. The drawback of MPPI is that the resulting optimal decisions are noisy, and this can result in undesirable stochastic behavior and slow convergence.

Recent efforts to mitigate the negative effect of noise include sampling from colored noise (Vlahov et al. 2024) to have temporal correlation for a smoother trajectory, and using Hessian of cost as sampling covariance for improving convergence (Yi et al. 2024). Furthermore, techniques such as spline parameterization of control can effectively reduce control jitter (Miura et al. 2024; Xue et al. 2025). Although they all alleviate the negative effect, often do so at the cost of expressive flexibility or by reintroducing differentiability requirements.

While the sampling aspect of SO provides a way to handle complex objective functions, this feature does not completely address the issue. Naive sampling can often be inefficient in exploring the state space. These limitations motivated the work on multimodality. One common strategy involves a coarse global search followed by local refinement. The method proposed by Osa (2020) utilizes a Gaussian Mixture Model (GMM) to coarsely fit multiple candidate solutions in a reduced basis-function space, followed by a gradient-based optimization to refine them. Although this approach works well in a robotic arm as presented, it is fundamentally kinematic in nature. Because the first sampling phase is decoupled with the system dynamics, it would become a bottleneck for underactuated systems where the manifold of feasible trajectories is much more strict. Sundaralingam et al. (2023) adapts this concepts for the GPU era by adding massive parallelization. It uses MPPI-like stage to seed a kinematic gradient-based optimizer. Kim et al. (2025) runs MPPI to produce coarse but diverse trajectory, then refines it with gradient based interior-point DDP solver.

Similar, but another way to combine the two optimization techniques, i.e., gradient and sampling, to handle multimodality is to perturb a single-seed gradient-based optimization via sampling. In Schulman et al. (2014), SQP-based trajectory optimizer is perturbed by randomly sampled noise during optimization to find better local solutions. Authors in Zucker et al. (2013) perturb gradient-based kinematic optimizer by sampling momentum from energy-like distribution induced by the cost of the trajectory to jump to a better local solution.

Beyond purely optimization-based methods, recent research has explored the use of generative models to provide multimodal priors for planning. For instance, Urain et al. (2022) utilize an energy-based model to represent multimodal distributions, which are then integrated into gradient-based and stochastic optimizers. Similarly, Carvalho et al. (2023) employ a diffusion model to capture complex trajectory priors from large-scale datasets. Authors in Huang et al. (2024) leverage a generative prior to initialize the parallelized optimization mentioned earlier (Sundaralingam et al. 2023) in an informed way. While these methods excel at representing the multimodality present in historical data, they are fundamentally data-dependent.

Le et al. (2023) utilizes Optimal Transport to handle multimodality, they often rely on a simplified linear Gaussian transition to enforce smoothness. This formulation implicitly treats dynamics as a soft smoothness penalty rather than following the true nonlinear dynamics. While Pan et al. (2024) adopts the nomenclature of generative diffusion, its underlying mechanism is a multi-step adaptation of the CEM using a noise-annealing schedule.

Alternative algorithms Lambert et al. (2021); Lambert and Boots (2021); Power and Berenson (2024) explore multimodality by making use of Stein Variational Gradient Descent (SVGD) (Liu and Wang 2016). SVGD is a particle-based optimization method that actively maintains solution diversity by using a kernel-based repulsive force. The work in Lambert et al. (2021); Lambert and Boots (2021) can be seen as SV extensions of MPPI. The method is also applied in kinematic planners, such as (Yin et al. 2025), and as a parallelized search algorithm (Lee et al. 2024). In Power and Berenson (2024); Lee et al. (2025), a constrained SV TO algorithm is proposed. However, the approach scales unfavorably with the dimensionality of control, state variables, time horizon, and the number of equality/inequality constraints. In Barcelos et al. (2024), the authors proposed signature-based repulsion, which, unlike pointwise diversity metrics that often yield redundant temporal variations of a single path, ensures that exploration occurs over topologically distinct and geometrically meaningful trajectories.

Motivated by the complementary strengths of these methods, we propose a framework that couples sampling- and gradient-based optimization. Specifically, we investigate two mechanisms that combine sampling- and gradient-based optimization to address the challenges of MPC in non-convex landscapes. One of the primary objectives is to provide a more rigorous analytical understanding of how these hybrid structures prevent optimization from becoming trapped in sub-optimal local minima.

The first mechanism is a schedule of optimization steps consisting of several Newton/second-order steps followed by an iteration that relies on sampling. The motivation for this hybrid process is that while Newton steps can locally optimize the objective function, the sampling phase promotes the escape of the trajectory from poor local minima. In the context of TO for robotics, this mechanism was first introduced in Maximum Entropy DDP (ME-DDP) (So et al. 2022) with Shannon's entropy and then extended to the broader class of Tsallis entropy (Aoyama and Theodorou 2024) in unconstrained settings. It is also interesting to note

that this hybrid optimization mechanism was also proposed in the area of business administration and management science [Harford \(2016\)](#) as a mental model to enhance creativity and improve innovation.

The second mechanism provides a further extension of ME-DDP through Stein Variational DDP (SV-DDP). Based on the Stein Variational Newton's Method (SVNM) ([Detommaso et al. 2018](#)), SV-DDP is a kernel-based extension of ME-DDP. In SV-DDP, the optimization process alternates between two phases as in ME-DDP. Starting from different initial trajectories, the first phase consists of multiple DDP updates performed in parallel fashion, resulting in optimized state and control trajectories. In the second phase, the control trajectories are updated with the rule based on SVNM to generate the new state trajectories. This phase facilitates exploration by pushing the trajectories apart using a kernel-based repulsion force.

A preliminary version of this work appeared in [Aoyama et al. \(2025\)](#). This journal version expands on the previous work by providing a unified mathematical derivation of the algorithm family, a rigorous interpretation of exploration via Hessian, extensive simulation-based comparisons, and hardware validation.

2 Background

In this section, we establish theoretical preliminaries for dynamic optimization and entropic regularization. We first formalize the entropic regularization of a dynamic optimization, followed by a review of DDP and ME-DDP. To align with standard conventions in information-theoretic literature, we denote the natural logarithm as $\ln$ throughout this section.

2.1 Entropic-regularized Dynamic Optimization

With the deterministic dynamics $x_{t+1} = f(x_t, u_t)$, with state $x \in \mathbb{R}^{n_x}$ and control $u \in \mathbb{R}^{n_u}$, we consider the following TO problem:

$$\min_{X,U} J(X,U) = \min_{X,U} \sum_{t=1}^{T-1} l(x_t, u_t) + \Phi(x_T), \quad (1)$$

subject to the dynamics. Here, we define state and control trajectories as $X = [x_1, \dots, x_T]$ and $U = [u_1, \dots, u_{T-1}]$. The scalar-valued functions $l(\cdot, \cdot)$, $\Phi(\cdot)$, and $J(\cdot, \cdot)$ denote the running, terminal, and total cost of the problem, respectively. There exist several second-order solvers that can solve the problem above. Typically, they solve Quadratic Programming (QP) with a quadratic approximation of the cost under constraints from dynamics. Two well-known classes of algorithms are DDP ([Jacobson and Mayne 1970](#)) and SQP ([Gill et al. 2000](#)). With quadratic approximation of dynamics, DDP effectively splits the problem into a sequence of stage-wise subproblems and solves them efficiently. SQP solves a large QP over trajectories under linearized dynamics, although there exist methods to solve QP efficiently with LQR [Rao et al. \(1998\)](#). Both of these methods are successfully used in robotic applications. However, since they are relying on local information on the cost and dynamics, i.e., gradient and Hessian, they are

vulnerable to being trapped at a poor local solution as mentioned in [Introduction](#).

To alleviate the issue, we consider a stochastic control policy $\Pi(U|X)$ and introduce expected entropy of the policy:

$$\begin{aligned} \mathcal{H}[\Pi] &= \mathbb{E}_{P(X)} [\mathbb{E}_{\Pi(U|X)} [-\ln \Pi(U|X)]] \\ &= -\mathbb{E}_{P(X,U)} [\ln \Pi(U|X)], \end{aligned}$$

where $P(X)$ is the marginal distribution of the state trajectory induced by the policy Π , and $P(X,U) = P(X)\Pi(U|X)$ is the resulting joint distribution of the state and control trajectories. Here, we use the term expected entropy because it represents the conditional entropy of the control trajectory, marginalized over the state trajectory distribution. We add it to the objective to promote exploration. Incorporating the dynamics and taking expectation over a trajectory, we have a new objective:

$$J_\Pi(x_1, \Pi) = \mathbb{E}_{P(X,U)} \left[\sum_{t=1}^{T-1} l(x_t, u_t) + \Phi(x_T) \right] - \tau \mathcal{H}[\Pi]. \quad (2)$$

The temperature parameter τ acts as a thermodynamic scaling factor. The intuition here is that, at high temperatures, the control (thermodynamic particles) has high kinetic energy, spreading across the cost landscape to maximize entropy. This prevents the optimizer from collapsing into a local minimum. In contrast, when the temperature is low, the control acts like a crystal, which does not explore the landscape.

With the objective with the entropy, the TO problem seeks a stochastic policy that minimizes the original objective while maximizing the corresponding entropy. The resulting policy can explore multiple local solutions and is robust to being captured by poor ones. There exist multiple approaches to solve the problem. One of the simplest cases is with a feedforward control $\Pi(U|x_1)$ which gives

$$\pi^*(U|x_1) = Z_0^{-1} \exp[-J(x_1, U)/\tau], \quad (3)$$

where Z_0 is a partition function [Liu et al. \(2017\)](#). In this work, we use DDP to effectively obtain and utilize the feedback policy.

2.2 Differential Dynamic Programming

In this section, we review DDP. Although this is a classic work, we highlight its inherent quadratic structure and treatment of its Hessian. This is because the Hessian plays a key role in the exploration strategies developed in the following sections.

We consider the cost-to-go at time step $t = i$, i.e., cost starting from $t = i$ to N , denoted by $J_i(X_i, U_i)$. This is given by

$$J_i(X_i, U_i) = \left[\sum_{t=i}^{T-1} l_t(x_t, u_t) \right] + \Phi(x_T)$$

with trajectories starting from time step i : $X_i = [x_i, \dots, x_N]$, $U_i = [u_i, \dots, u_{N-1}]$. The value function is defined as the minimum cost-to-go in each state and time step t via $V_t(x_t) := \min_{u_t} J_t(X_t, U_t)$. Given Bellman's principle of optimality that provides the following rule

$$V_t(x_t) = \min_{u_t} \underbrace{[l(x_t, u_t) + V_{t+1}(x_{t+1})]}_{Q_t(x_t, u_t)}, \quad (4)$$

where $Q_t(x_t, u_t)$ is action-state, or simply Q function, DDP finds local solutions to the minimization of (4) by expanding about nominal trajectories $\bar{X}$ and $\bar{U}$.

The first step of the minimization is to perform quadratic expansions of Q_t about nominal pair $(\bar{x}_t, \bar{u}_t)$ with the deviation $\delta x_t = x_t - \bar{x}_t$, $\delta u_t = u_t - \bar{u}_t$, obtaining

$$Q(x_t, u_t) \approx Q(\bar{x}_t, \bar{u}_t) + Q_u^\top \delta u_t + Q_x^\top \delta x_t + \frac{1}{2} \delta u_t^\top Q_{uu} \delta u_t + \delta u_t^\top Q_{ux} \delta x_t + \frac{1}{2} \delta x_t^\top Q_{xx} \delta x_t, \quad (5)$$

where we drop the time index t for Q . For readability, we drop the time subscript t where the time dependency can be recovered from the arguments or associated variables (e.g., writing $Q_u^\top \delta u_t$ instead of $Q_{u,t}^\top \delta u_t$), hereafter.

The quadratic approximation is a standard process in nonlinear optimization, as used in Newton's method (Nocedal and Wright 2006).

Assuming that the Hessian Q_{uu} is Positive Definite (PD), we can explicitly optimize Q approximated in (5) with respect to δu_t by computing a partial derivative of (5) with respect to δu_t and setting it zero. This minimization yields the following local optimal control law

$$\delta u_t^* = \kappa_t + K_t \delta x_t, \quad (6)$$

$$\text{with } \kappa_t = -Q_{uu}^{-1} Q_u, \quad K_t = -Q_{uu}^{-1} Q_{ux},$$

where κ and K are known as feedforward and feedback gains, respectively.

DDP has backward and forward passes. In the backward pass, the derivatives of Q functions which is later introduced in (8), gains in (6) are computed backward in time. In the forward pass, the new control $\bar{u}_t + \delta u_t^*$ is propagated forward in time to give a new pair of nominal trajectories.

2.2.1 Backward pass To obtain the derivatives of Q functions that are required to compute gains, we perform a similar expansion on the term $l(x_t, u_t) + V_{t+1}(x_{t+1})$ given in the definition of Q function in (4), and eliminate δx_{t+1} using quadratic approximation of the dynamics:

$$\delta x_{t+1} \approx f_x \delta x_t + f_u \delta u_t + \frac{1}{2} \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}^\top \begin{bmatrix} f_{xx} & f_{xu} \\ f_{ux} & f_{uu} \end{bmatrix} \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}, \quad (7)$$

where f_x and f_u denote the state and control Jacobians, while the block-matrix in the last term contains the Hessian tensors. Specifically, f_{xx} and f_{uu} represent the second-order sensitivities with respect to state and control, respectively, and f_{xu} accounts for their coupling. Mapping the terms on both sides of quadratic approximation of $Q(x_t, u_t) = l(x_t, u_t) + V_{t+1}(x_{t+1})$, we obtain the derivatives of Q (evaluated on $\bar{X}$ and $\bar{U}$) as follows.

$$Q_{u,t} = l_u + f_u^\top V_{x,t+1}, \quad Q_{x,t} = l_x + f_x^\top V_{x,t+1}, \quad (8)$$

$$Q_{uu,t} = l_{uu} + f_u^\top V_{x,t+1} f_u + f_{uu} \cdot V_{x,t+1},$$

$$Q_{xu,t} = l_{xu} + f_x^\top V_{x,t+1} f_u + f_{xu} \cdot V_{x,t+1},$$

$$Q_{xx,t} = l_{xx} + f_x^\top V_{x,t+1} f_x + f_{xx} \cdot V_{x,t+1},$$

where derivatives of the running cost and dynamics are evaluated at time t , and $\cdot$ for Hessians and $V_{x,t+1}$ is tensor contraction along the first (state) axis.

Now, δu_t^* is computed using $V_{x,t+1}$ and $V_{xx,t+1}$, which are in one time step ahead. To propagate derivatives of $V_t(x)$ back in time, we consider the quadratic expansion of $V_t(x)$, that is,

$$V(x_t) = V(\bar{x}_t) + V_x(\bar{x}_t)^\top \delta x_t + \frac{1}{2} \delta x_t^\top V_{xx}(\bar{x}_t) \delta x_t,$$

and equate the equation with the quadratic expansion of Q in (5) through (4). Since we now have a solution of the $\min_{u_t}$ in the right-hand side of (4), by substituting δu^* for δu , the min operator vanishes. This allows us to compare the coefficients of δx_t by mapping the terms, giving the backward recursions:

$$V_{x,t} = Q_{x,t} + K_t^\top Q_{uu,t} \kappa_t + K_t^\top Q_{u,t} + Q_{ux,t}^\top \kappa_t, \quad (9)$$

$$V_{xx,t} = Q_{xx,t} + K_t^\top Q_{uu,t} K_t + K_t^\top Q_{ux,t} + Q_{ux,t}^\top K_t,$$

with the terminal condition

$$V_T(x_T) = \Phi(x_T), \quad V_{x,T} = \Phi_x(x_T), \quad V_{xx,T} = \Phi_{xx}(x_T). \quad (10)$$

We note that in our implementation, we drop the second-order information of the dynamics and use a linear approximation, which corresponds to iterative LQR (Li and Todorov 2004).

2.2.2 Forward pass In the forward pass, the new control $\bar{u}_t + \delta u_t^*$ is propagated forward in time to give a new pair of nominal trajectories, typically with a backtracking line search to absorb the discrepancy between the quadratic cost model, linear or quadratic dynamics model, and the actual ones.

2.2.3 Regularization To compute the optimal gains by minimizing (5), the Hessian Q_{uu} must be PD. Furthermore, established convergence analysis relies on PD Q_{uu} or on the convexity of the surrogate model formed with regularized Q_{uu} (Liao and Shoemaker 1991; Liao 1996). When the Q_{uu} is not PD, it must be regularized. One of the well-known strategies is

$$Q_{uu,t}^{\text{reg}} = Q_{uu,t} + \mu_{\text{reg}} I_{n_u}, \quad (11)$$

which is equivalent to adding a cost penalizing large δu_t via a quadratic trust-region penalty (Liao and Shoemaker 1991).

In practical implementations of DDP, if cost reduction is not achieved with a small step size in the forward pass, the backward pass is rerun with a larger regularization parameter (Tassa et al. 2012).

This effectively restricts the optimization to a smaller, more reliable neighborhood of the nominal trajectory where the local approximation remains valid, and therefore, cost reduction is expected.

2.3 Maximum Entropy Differential Dynamic Programming

This section provides a review of ME-DDP and demonstrates its efficacy in solving the entropy-regularized dynamic optimization problem. We also introduce its unimodal and multimodal policies. We have a detailed derivation in the Appendix.

Using the Markovian assumption Puterman (1994), we decompose the policy (3) defined over the trajectories into

multiple of policies at each time step:

$$\Pi(U|X) = \prod_{t=1}^{T-1} \pi(u_t|x_t).$$

Then, the expected total entropy of the policy $\mathcal{H}[\Pi]$ becomes the sum of stage-wise entropy:

$$\begin{aligned} \mathcal{H}[\Pi] &= \mathbb{E}_{P(X,U)} \left[-\ln \prod_{t=1}^{T-1} \pi(u_t|x_t) \right] \\ &= \sum_{t=1}^{T-1} \mathbb{E}_{P(x_t, u_t)} [-\ln \pi(u_t|x_t)] \\ &= \sum_{t=1}^{T-1} \mathbb{E}_{P(x_t)} [\mathbb{E}_{u_t \sim \pi(\cdot|x_t)} [-\ln \pi(u_t|x_t)]] \\ &= \sum_{t=1}^{T-1} \mathbb{E}_{P(x_t)} [\mathcal{H}[\pi(\cdot|x_t)]] , \end{aligned}$$

With this decomposition, we consider the stage-wise minimization of (2) given state x_t ,

$$\min_{\pi} \{ \mathbb{E}_{u \sim \pi(\cdot|x)} [l(u_t, x_t) + \tilde{V}_{t+1}(x_{t+1})] - \tau \mathcal{H}[\pi(\cdot|x_t)] \}, \quad (12)$$

$$\text{subject to } \int \pi(u_t|x_t) du = 1,$$

which can be seen as an entropic regularized version of (4). Here, we use $\tilde{V}$ to denote the value function of the entropic regularized problem. The minimization in (12) results in the optimal control policy π_t^* :

$$\pi^*(u|x) = Z(x)^{-1} \exp[-\tilde{Q}(x, u)/\tau], \quad (13)$$

$$\text{with } \tilde{V}(x) = -\tau \ln Z(x), \quad (14)$$

where $Z(x)$ is the corresponding partition function and $\tilde{Q}_t(x_t, u_t) = l(x_t, u_t) + \tilde{V}_{t+1}(x_{t+1})$. The relationship represents a smooth approximation of the Bellman optimality operator. The intuition here is that, by exponentiating the negative Q function, the lowest value are magnified relative to higher ones. The subsequent operation with $\ln$ and negation returns the result to the original scale. This operation effectively acts as a Soft-min operator (Levine 2018; Ziebart 2010).

2.3.1 Unimodal Gaussian Policy Combined with the quadratic approximation of Q function (5) utilized in DDP, (13) indicates that the optimal policy has a form of unimodal Gaussian $\pi^*(\delta u|\delta x) \sim \mathcal{N}(\delta u^*, \tau Q_{uu}^{-1})$, where δu^* is a solution of deterministic DDP in (6).

2.3.2 Multimodal Gaussian Policy In the multimodal case, we consider N trajectories or modes:

$$X^{(n)} = [x_1^{(n)}, \dots, x_T^{(n)}], \quad U^{(n)} = [u_1^{(n)}, \dots, u_{T-1}^{(n)}],$$

with $n = 1, \dots, N$ and the LogSumExp approximation of the value function

$$\tilde{V}(x) = -\tau \ln \sum_{n=1}^N \exp(-\tilde{V}^{(n)}(x)/\tau),$$

where the superscript (n) denotes the n th trajectory. The exponential transformation $\mathcal{E}_\tau(y) = \exp(-y/\tau)$ of the value function results in a control policy represented as a mixture of Gaussians whose categorical distribution is proportional to the value function of each trajectory. This multimodal policy is represented as follows:

$$\begin{aligned} \pi^*(u|x) &= \sum_{n=1}^N \omega^{(n)}(x) \pi^{*(n)}(u|x) \\ \text{with } \omega^{(n)} &\propto \mathcal{E}_\tau[\tilde{V}^{(n)}(x)], \quad \text{and} \quad \sum_{n=1}^N \omega^{(n)} = 1. \end{aligned} \quad (15)$$

The intuition here is that the policy decides which modes to sample based on the value function and then samples from the corresponding Gaussian. We note that while (15) theoretically allows the policy to be multimodal, in practice these modes may collapse into a single dominant local minimum. This numerical collapse significantly diminishes the exploratory benefit of ME-DDP.

To mitigate the issue, we introduce a heuristic in our implementation with a lower bound $\omega_{\min}$ on the weights. The weights are updated as $\hat{\omega}^{(n)} = \max(\omega^{(n)}, \omega_{\min})$ and subsequently renormalized.

2.3.3 Exploration (sampling) vs Exploitation (gradient)

In the ME-DDP framework, the entropic term facilitates exploration, while the deterministic DDP part governs exploitation. As the title suggests, excessive exploration can wash out the signal provided by the gradient. If the temperature τ is too high, the optimizer may not identify the underlying cost structure, leading to poor convergence. To mitigate this, the authors of So et al. (2022) suggest that sampling should not happen at all iterations. Instead, perturbations are introduced periodically to allow the nominal trajectory to stabilize between exploratory phases. Although the original work was in the unconstrained setting, we observe that this is even more important when constraints are handled via DDP. Furthermore, following the spirit of Dong and Tong (2021), we implement a heuristic that preserves the trajectory with the best metric without sampling. This approach ensures that the high-precision convergence properties of the deterministic DDP are maintained for the current mode.

3 Stein Variational Dynamic Optimization

In this section, we introduce SV-DDP, a kernel extension of the ME-DDP framework. By incorporating a kernel-based repulsive force, the algorithm actively prevents particle collapse and preserves a diverse set of modes during the exploration process.

To provide the theoretical basis, we first review SVGD and its connection to functional gradient descent. We then discuss the SVNMM method as a second-order extension. Subsequently, we apply these techniques to the DDP framework to derive SV-DDP. Finally, we discuss the essential components of the algorithm.

3.1 Stein Variational Gradient Descent and Newton's Method

SVGD minimizes the Kullback-Leibler (KL) divergence between a set of particles and a target distribution by performing functional gradient descent in a Reproducing Kernel Hilbert Space (RKHS). SVNM incorporates the Hessian to accelerate convergence and better capture the geometry of the underlying distribution.

For comprehensive treatments, we refer the reader to [Liu and Wang \(2016\)](#); [Liu \(2017\)](#) for SVGD and to [Detommaso et al. \(2018\)](#) for SVNM. In this subsection, $x \in \mathbb{R}^d$ denotes a generic optimization variable rather than the system state, and $x_n \in \mathbb{R}^d$ represents a specific sampled point (particle).

3.1.1 Stein Variational Gradient Descent Let p on $\mathbb{R}^d$ be a target distribution that we wish to approximate using a collection of samples. We specify the argument of $p(\cdot)$, when we evaluate it at a point x_n as $p(x_n)$. With samples $\{x_n\}$ from a tractable reference distribution q on $\mathbb{R}^d$, SVGD iteratively computes a transport map $\mathcal{T} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ so that the transformed samples of q , i.e., $\{\mathcal{T}(x_n)\}$ can empirically approximate p . This map is obtained by solving the following optimization problem:

$$\min_{\mathcal{T}} \{D_{\text{KL}}(q^{l+1} \| p)\} = \min_{\mathcal{T}} \left\{ \mathbb{E}_{\mathcal{T}_{\#} q^l} \left[\ln \left(\frac{\mathcal{T}_{\#} q^l}{p} \right) \right] \right\}, \quad (16)$$

where $\mathcal{T}_{\#} q^l = q^{l+1}$ and l stands for l -th iteration. Here, D_{KL} is KL divergence that measures the difference between the two distributions. To simplify this problem, SVGD considers the vector-valued Reproducing Kernel Hilbert Space (RKHS) $\mathcal{F}^d = \mathcal{F} \times \cdots \times \mathcal{F}$, where $\mathcal{F}$ is a scalar-valued RKHS with kernel $k(x, x')$. This framework allows us to represent functions as weighted compositions of kernels centered at the samples. Furthermore, it also allows us to solve the optimization problem in functional space by solving the corresponding problem with the weights using standard optimization techniques. To solve (16), we define the map $\mathcal{T}$ as a perturbation $\mathcal{P}$ of the identity map I in $\mathcal{F}^d$ as

$$\mathcal{T}^l(x) = I(x) + \mathcal{P}(x), \quad \text{for } \mathcal{P}(x) \in \mathcal{F}^d.$$

With the map, we reformulate the objective of the problem (16) as

$$\hat{J}_{q^l}[\mathcal{P}] = D_{\text{KL}}((I + \mathcal{P})_{\#} q^l \| p). \quad (17)$$

To minimize the objective $\hat{J}_{q^l}[\mathcal{P}]$, we consider a perturbation $\mathcal{P}$ in the direction of the functional gradient. Specifically, we define the descent direction as the negative functional gradient of $\hat{J}_{q^l}$ evaluated at the identity map (represented by the zero perturbation $\mathbf{0}$):

$$\mathcal{T}^l = I - \alpha \nabla_{\mathcal{P}} \hat{J}_{q^l}[\mathcal{P}]|_{\mathcal{P}=\mathbf{0}}$$

where $\nabla_{\mathcal{P}} \hat{J}_{q^l}[\mathcal{P}]$ denotes the functional derivative of the objective with respect to $\mathcal{P}$. This formulation allows us to treat the transformation of the distribution as a steepest descent process in the space of maps. By selecting $\mathcal{P} = -\alpha \nabla_{\mathcal{P}} \hat{J}_{q^l}[\mathbf{0}]$, we shift the current distribution q^l towards the target p in the direction that yields the most rapid decrease

in KL divergence. This construction provides a direct link between standard gradient descent on a point x and the functional update of the entire distribution q .

The descent direction satisfies the following condition with $S \in \mathcal{F}^d$:

$$D\hat{J}_q[S](\mathcal{V}) = \langle \nabla \hat{J}_q[S], \mathcal{V} \rangle_{\mathcal{F}^d}, \quad \forall \mathcal{V} \in \mathcal{F}^d,$$

where we drop l . Here, the left-hand side is the first variation of $\hat{J}_q$ at S along $\mathcal{V}$ defined as follows:

$$D\hat{J}_q[S](\mathcal{V}) = \lim_{\zeta \rightarrow 0} (\hat{J}_q[S + \zeta \mathcal{V}] - \hat{J}_q[S]) / \zeta.$$

The authors of [Liu and Wang \(2016\)](#) showed that the functional gradient at $\mathbf{0}$ is empirically approximated by N particles:

$$\begin{aligned} & -\nabla \hat{J}_q[\mathbf{0}](x_i) \\ &= \frac{1}{N} \sum_{n=1}^N \left[(\nabla_{x_n} \log p(x_n)) k(x_n, x_i) + \nabla_{x_n} k(x_n, x_i) \right], \end{aligned} \quad (18)$$

where the first term in the summation follows the gradient direction, and the second term spreads the particles apart from each other. Thus, it is known as the repulsive force. Finally, the update rule for samples is given by setting $\phi^* = -\nabla \hat{J}_q[\mathbf{0}]$ and with a step size α as follows:

$$x_n^{l+1} \leftarrow x_n^l + \alpha \phi^*(x_n^l).$$

3.1.2 Stein Variational Newton Method SVGD constructs a vector field and evaluates it at a single point to determine how a particle moves. SVNM additionally incorporates second-order information by approximating the Hessian of the KL functional, which is an operator rather than a vector. Operators act on functions and therefore require two evaluation points. To reduce indices, we adopt the following notation: z represents the point where the input function is evaluated (input location), and y represents the location where the output of the operator is evaluated (output location). Let us consider a function f_0 and an operator H_0 , then applying the operator to the function gives

$$(H_0 f_0)(y) = \int k(y, z) f_0(z) dz,$$

where the kernel $k(y, z)$ defines the coupling of z and y .

For second-order optimization in functional space, we define the second variation of $\hat{J}_q$ at $\mathbf{0}$ along the pair of directions $\mathcal{V}, W \in \mathcal{F}^d$ as:

$$D^2 \hat{J}_q[\mathbf{0}](\mathcal{V}, W) = \lim_{\zeta \rightarrow 0} (D\hat{J}_q[\zeta W](\mathcal{V}) - D\hat{J}_q[\mathbf{0}](\mathcal{V})) / \zeta.$$

The Newton direction W is obtained by the optimality condition given by the following equation.

$$D^2 \hat{J}_q[\mathbf{0}](\mathcal{V}, W) = -D\hat{J}_q[\mathbf{0}](\mathcal{V}), \quad \forall \mathcal{V} \in \mathcal{F}^d,$$

which gives the transformation map of the Newton direction as a perturbation of the identity map as $\mathcal{T} = I + \alpha W$. The authors in [Detommaso et al. \(2018\)](#) proved that the Newton direction $W = (w_1, \dots, w_d)^T$ satisfies for all $\mathcal{V} =$

$$(v_1, \dots, v_d)^\top \in \mathcal{F}^d,$$

$$\sum_{i=1}^d \left\langle \sum_{j=1}^d \langle h_{ij}(y, z), w_j(z) \rangle_{\mathcal{F}} + \nabla \hat{J}[\mathbf{0}]_i(y), v_i(y) \right\rangle_{\mathcal{F}} = 0,$$

with $h_{ij}(y, z) = \mathbb{E}_{x \sim q} \left[-[\nabla^2 \log p(x)]_{i,j} k(x, y) k(x, z) + \nabla k(x, y)_i \nabla k(x, z)_j \right].$ (19)

In the same work, the Galerkin approximation of the solution of W is also proposed, where W is expanded on $\mathcal{F}^d = \text{span}\{k(x_1, \cdot), \dots, k(x_N, \cdot)\}$. The expansion leads to the following approximation with coefficients $\beta^n \in \mathbb{R}^d$

$$w(z) = \sum_{n=1}^N \beta^n k(x_n, z), \quad (20)$$

where the superscripts n indicate the n -th particle. The coefficients β s are given as a solution of linear systems:

$$\sum_{n=1}^N H^{s,n} \beta^n = -\nabla \hat{J}_p[\mathbf{0}](x_s), \quad \text{for } s = 1, \dots, N,$$

where the Hessian is denoted as $H_{i,j}^{s,n} = h_{ij}(x_s, x_n)$.

The authors also propose the block-diagonal approximation of the system for parallelization, by transforming the system as:

$$H^{n,n} \beta^n = -\nabla \hat{J}_p[\mathbf{0}](x_n), \quad \text{for } n = 1, \dots, N, \quad (21)$$

which means that the off-diagonal blocks are approximated to zero $H^{s,n} = O_d$ for $s \neq n$, where $O_d \in \mathbb{R}^{d \times d}$ is a zero matrix. A detailed explanation with an example is provided in the appendix. SVNМ repeats the process of solving the linear systems above and updating particles with

$$x_n^{l+1} \leftarrow x_n^l + \alpha w(x_n^l),$$

to approximate the optimal distribution.

3.2 ME-DDP with Functional Gradient and Hessian

In this section, we apply SVNМ to ME-DDP to derive a new algorithm SV-DDP. The motivation here is that, by using a kernel-based repulsive force, the algorithm can keep trajectories representing the modes diverse. This mechanism prevents mode collapse and maintains high exploration capability.

We consider N trajectories optimized by DDP to compose policy as in the MG-ME-DDP. Here, we assume that the global Q is approximated by $Q^{(n)}$ around the trajectories $[x^{(n)}, u^{(n)}]$.

We initially considered a GMM to approximate Q . However, since each DDP trajectory is local in nature, treating the ensemble as a global PDF via GMM interpolation is physically inconsistent. Such an approach would require evaluating local policy in regions far from their nominal trajectories, where the underlying quadratic approximations are not valid.

By substituting the optimal policy $\pi^*(u)$ for p in the functional gradient (18), we obtain

$$\begin{aligned} & -\nabla \hat{J}_q[\mathbf{0}](u^{(s)}) \\ &= \frac{1}{N} \sum_{n=1}^N \left[\nabla_{u^{(n)}} \left(-\frac{Q(x^{(n)}, u^{(n)})}{\tau} \right) k(u^{(s)}, u^{(n)}) \right. \\ & \quad \left. + \nabla_{u^{(n)}} k(u^{(n)}, u^{(s)}) \right], \end{aligned}$$

for $s = 1, \dots, N$. We consider the quadratic approximation of Q function and deviation of trajectories. Due to the linearity of the deviation, derivative with respect to u is now equivalent to that of δu . As in the case of ME-DDP, the optimizer alternates optimization and exploration. Here, we assume that DDP refinement can bring the trajectory to a locally convex region with PD Q_{uu} , and consider the optimal control u^* computed by DDP. By substituting the optimal control back into the quadratic approximation of Q , we have

$$\begin{aligned} & -\nabla \hat{J}_q[\mathbf{0}](u^{*(s)}) \\ &= \frac{1}{N} \sum_{n=1}^N \left[-\frac{1}{\tau} \underbrace{(Q_u^{(n)} + Q_{ux}^{(n)} \delta x^{(n)} + Q_{uu}^{(n)} \delta u^{*(n)})}_{\approx 0} \right. \\ & \quad \left. \times k(u^{*(s)}, u^{*(n)}) + \nabla_{u^{*(n)}} k(u^{*(n)}, u^{*(s)}) \right], \end{aligned} \quad (22)$$

where the derivatives of Q are evaluated at $\bar{u}^{(n)}$. The coefficient of the kernel $k(\cdot, \cdot)$ is zero because of the optimality condition in the derivation of DDP. Thus, we are left with the repulsive force terms. This seems attractive for spreading trajectories. However, since the temperature τ is lost, this formulation cannot capture the relative importance of the entropy in the objective.

To recover the temperature, we use SVNМ. By substituting $\pi^*(u)$ for $p(x)$ in the approximated Hessian (21) with the terms in (19), and performing empirical approximation, the Hessian for SV Newton's method is obtained as:

$$\begin{aligned} H^{s,s} &= \frac{1}{N} \sum_{n=1}^N \left[\frac{Q_{uu}^{(n)}}{\tau} k(u^{(n)}, u^{(s)})^2 \right. \\ & \quad \left. + \nabla_{u^{(n)}} k(u^{(n)}, u^{(s)}) \nabla_{u^{(n)}} k(u^{(n)}, u^{(s)})^\top \right], \end{aligned} \quad (23)$$

where we drop optimality $*$ for readability. By plugging this into (21) and solving the equations, we get the SV update (or sample) of control:

$$u^{(s)} \leftarrow u^{(s)} + \alpha w(u^{(s)}) + K^{(s)} \delta x^{(s)} \quad (24)$$

$$\text{with } w = \sum_{n=1}^N \beta^n k(u^{(n)}, u^{(s)}),$$

where K is feedback gain of DDP (6).

Consequently, the temperature τ is naturally recovered within the Hessian $H^{s,s}$. We therefore propose that this approach is the consistent formulation for incorporating SV methods into the DDP framework. In (23), the second term is an expected outer product of the gradient, which captures information on the variation in all directions like its Hessian Trivedi et al. (2014). Thus, $H^{s,s}$ can be seen as a sum of the Hessians of the objective and kernels. We analyze

Algorithm 1: Entropic Regularized DDP

Input: x_1 : Initial state, $\bar{u}$: Initial nominal sequence
 Σ_0 : Initial covariance, N : Number of modes
 m : Sampling frequency, I : Max iterations
 I_{DDP} : DDP iterations per mode
Notation:
 U : Batched control trajectory $\{U^{(n)}\}_{n=1}^N$
 X : Batched state trajectory $\{X^{(n)}\}_{n=1}^N$
 Q_{uu} : Batched Hessian sequences $\{Q_{uu,1:T-1}^{(n)}\}_{n=1}^N$
 K : Batched feedback gain sequences $\{K_{1:T-1}^{(n)}\}_{n=1}^N$

Initialization

```

1  $U^{(1)} \leftarrow \bar{u}$ 
2 for  $n = 2$  to  $N$  in parallel do
3    $U^{(n)} \leftarrow \{u_t^{(n)} \sim \mathcal{N}(\bar{u}_t, \Sigma_0)\}_{t=1}^{T-1}$ 
4    $J^{(n)} \leftarrow \text{ComputeInitialCost}(X^{(n)}, U^{(n)})$ 
5 end
6  $b \leftarrow \arg \min_n \{J^{(n)}\}_{n=1}^N$       Best mode index

```

Optimization Loop

```

7 for  $i = 1$  to  $I$  do
8   if  $i \pmod m = 0$  then
9      $\{X, U\} \leftarrow \text{Exploration}(U, X, Q_{uu}, K, b)$ 
10    UG-ME-DDP: Algorithm B.1
11    MG-ME-DDP: Algorithm B.2
12    SV-DDP: Algorithm 2
13  end
14  Exploitation via Parallel DDP
15  for  $n = 1$  to  $N$  in parallel do
16     $X^{(n)}, U^{(n)}, K_{1:T-1}^{(n)}, Q_{uu,1:T-1}^{(n)}, J^{(n)} \leftarrow$ 
17     $\text{RunDDP}(X^{(n)}, U^{(n)}, I_{\text{DDP}})$ 
18  end

```

how the Hessian interacts with the repulsive force term in section [Inverse Hessian as covariance](#). We note that the SV framework is deterministic. In this method, stochasticity enters the system only via the random sampling used for initialization. Nevertheless, we conceptualize and refer to this behavior as an exploration mechanism, maintaining conceptual continuity with the ME-DDP framework.

We provide the schematic of ME-DDP variants in Fig. 1. We note that the update is not applied to the best trajectory, ensuring that at least one mode shows a monotonic cost reduction to preserve the convergence property of DDP as in [So et al. \(2022\)](#); [Dong and Tong \(2021\)](#). The proposed optimization framework, detailed in Algorithm 1, where an exploration mechanism based on the SV method is provided in 2. We have alternative mechanisms, i.e., UG- and MG-ME-DDPs in appendix.

3.3 Essential Components in SV-DDP

In this subsection, we first discuss the choice and properties of the kernel function, which serves as the core of the SV method and governs the behavior of the repulsive force. We

Algorithm 2: Stein Variational Exploration

Input: U, X, Q_{uu}, K, b
Output: Updated trajectories X, U

```

1 Compute Newton coefficients via (22) and (23)
2 for  $n = 1$  to  $N$  in parallel do
3   for  $t = 1$  to  $T - 1$  in parallel do
4     Solve  $H_t^{n,n} \beta_t^{(n)} = -\nabla \hat{J}_q[0](u_t^{(n)})$ 
5      $w_t^{(n)} \leftarrow \sum_{n'=1}^N \beta_t^{(n')} k(u_t^{(n)}, u_t^{(n')})$ 
6   end
7 end
8 Apply update to all modes except the current best
9 for  $n = 1$  to  $N, n \neq b$  in parallel do
10   $X^{(n)}, U^{(n)} \leftarrow$ 
11   $\text{LineSearch}(X^{(n)}, U^{(n)}, w_{1:T-1}^{(n)}, K_{1:T-1}^{(n)})$ 
12 end
13 return  $\{X, U\}$ ;

```

then proceed with the selection of the step size for the SV update rule.

3.3.1 Kernel and resampling As in other SV literature in robotics, we use the RBF kernel $k(x, x') = \exp(-\|x - x'\|^2/h)$ and choose its length parameter by median heuristic

$$h = \frac{\text{median}(\{\|u^{(i)} - u^{(j)}\|^2\}_{i < j})}{\log N}$$

([Garreau et al. 2018](#)). We note that while kernels such as the Inverse Quadratic (IQ) kernel are frequently used in static optimization and sampling to leverage heavy-tailed characteristics that induce repulsion at greater distances ([Gorham and Mackey 2017](#)), the RBF kernel remains dominant in robotics. Fig. 2 shows the RBF kernel and its derivatives. We argue that in high-dimensional robotics problems, local repulsion of RBF is more effective than far-field interaction of IQ.

Although the exponential decay of the RBF kernel is a fundamental property, it poses a challenge in high-dimensional spaces where the concentration of measure increases the average Euclidean distance between particles. In such regimes, the kernel value $k(u^{(i)}, u^{(j)})$, and consequently its gradient, tends to vanish, as the exponential decay outpaces the linear growth of the distance vector. This leads to a vanishing repulsive force, making the SV method less effective ([Zhuo et al. 2018](#)). The sum of local kernels alleviates this problem and is used in robotic applications ([Wang et al. 2021](#); [Lambert et al. 2021](#)). This is because, in these works, the input of the kernel is a full horizon of control sequence whose dimension is $n_u(T - 1)$. In our work, due to the stage-wise formulation of DDP, the input dimension is only n_u . Thus, it works well without the technique mentioned above.

The derivative of the kernel drives the repulsive force term. It reaches local extremum before decaying as the distance between points approaches zero. Consequently, while the force pushes trajectories apart at moderate distances, it vanishes when trajectories nearly coincide, potentially leading to mode collapse. To counteract this issue, we monitor the distance of the trajectories normalized

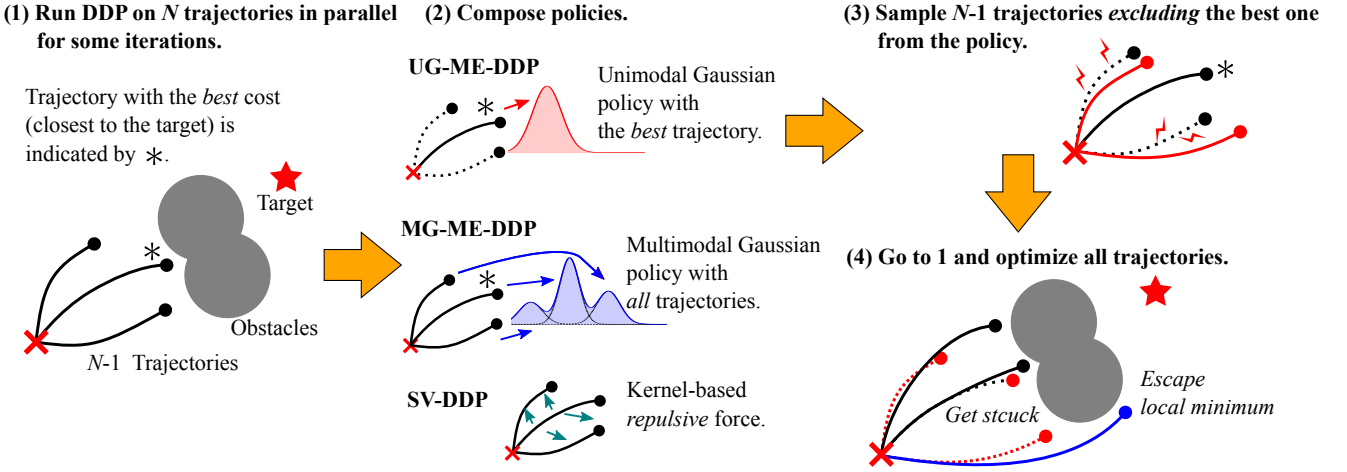


Figure 1. Schematic of algorithms in a trajectory optimization setting. (1) The optimizer initializes N trajectories, optimizing each via DDP for several iterations in parallel. The current best trajectory is marked ($*$). For illustrative clarity, we assume that the cost function is dominated by the distance to the target. (2) Policies are composed based on the specific algorithm: UG-ME-DDP uses a unimodal Gaussian centered on the best trajectory, MG-ME-DDP captures all trajectories in a multimodal distribution, and SV-DDP applies kernel-based repulsive forces to maintain diversity. Note that although the policies are constructed in control space, they are visualized here in state space for simplicity. (3) New trajectories are sampled from these policies for exploration, excluding the current best. (4) The cycle repeats. Although the best trajectory may get stuck at a poor local solution, the sampling mechanism and trajectories ran in parallel allows trajectories to be re-initialized into more promising areas to reach the target.

by the time horizon and dimension $\|u_i - u_j\| / \sqrt{T n_u}$. If the distance falls below a threshold, we keep the elite trajectory and resample the remaining redundant ones to ensure continuous exploration.

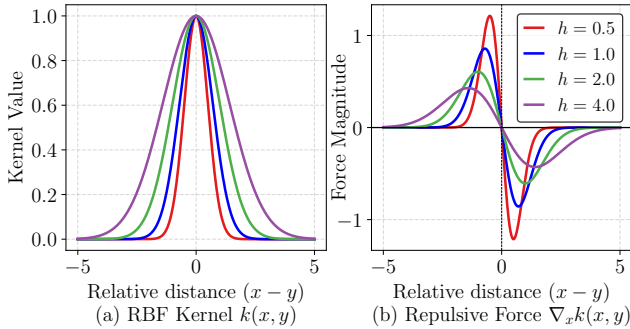


Figure 2. RBF kernel (a) and its gradient (b) across different bandwidths. The gradient represents the repulsive force of SV methods.

3.3.2 Step size selection Since the step size α determines the extent of the exploration, its choice is critical. While the SV literature often employs backtracking line search based on the Armijo condition (Chen et al. 2019; Chen and Ghattas 2020), robotic applications frequently tune task-specific fixed sizes (Wang et al. 2021; Lambert et al. 2021).

In our implementation, we observe that the repulsive force naturally pushes trajectories toward higher-cost regions, especially near obstacle boundaries. Consequently, a standard line search often prefers near zero α . To address this, we implement a cost-bounded heuristic: we accept the largest $\alpha \in (0, 1]$ that keeps the trajectory cost within a factor c_c (e.g., say 10-20) of the current minimum. This approach prioritizes global exploration while maintaining numerical stability.

4 Geometric Analysis and Algorithmic Synthesis

This section provides a formal justification for using the inverse Hessian as a covariance in sampling. Although utilizing Q_{uu}^{-1} , as a sampling covariance has been previously proposed in policy search literature (Levine and Koltun 2013), we present a more rigorous analysis to demonstrate why this choice is theoretically principled in the context of DDP. Subsequently, we provide how constraints are incorporated into our framework. Finally, we conclude with the implementation details for Model Predictive Control (MPC).

4.1 Inverse Hessian as covariance

To understand the efficacy of the sampling scheme, i.e., the inverse of the Hessian as the covariance, we first examine a simple quadratic function and its Hessian to establish the mathematical relationship. Then, we extend the intuition to a more general static optimization problem. The static examples here are highly relevant because DDP can be seen as a sequence of local static sub-problems. Therefore, the insights gained here apply directly to ME-DDP. Using the same example, we analyze the effect of Hessian regularization on the sampling covariance. This analysis is essential for practical applications. Because the underlying problem is non-convex, the optimizer must robustly handle non-PD Hessians while maintaining meaningful exploration. Finally, we discuss regularization itself.

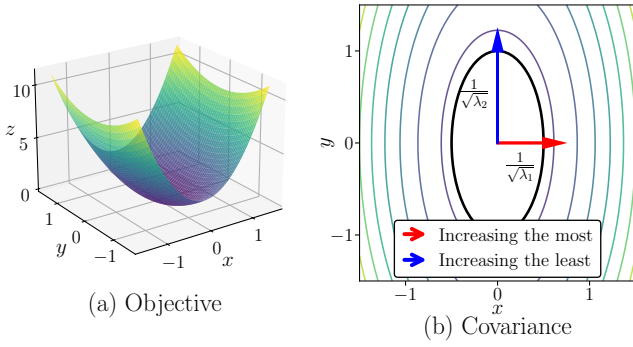


Figure 3. Surface plot of the quadratic function (a) and the resulting covariance ellipse overlaid on the contour plot (b). The blue and red arrows denote the principal directions of the covariance ellipse induced by the inverse of the Hessian H .

4.1.1 Property of Hessian We consider a two-dimensional function with a diagonal Hessian:

$$X^T H X, \text{ with } X = [x, y]^T \in \mathbb{R}^2, H = \text{diag}\{4, 1\}.$$

The surface plot and the covariance ellipse induced by the inverse of the Hessian (H^{-1}) are shown in Fig. 3. The Hessian has eigenvalues $\lambda_1 = 4$, $\lambda_2 = 1$ with corresponding eigenvectors $e_1 = [1, 0]^T$, $e_2 = [0, 1]^T$. This means that the function changes the most rapidly along e_1 and the least rapidly along e_2 . In the contour plot, the levels are denser along the x -direction, reflecting the higher curvature in that dimension.

When the Hessian is inverted, the eigenvalues become the reciprocals of the original ones, while the eigenvectors are preserved. Consequently, the eigenpairs are $(1/\lambda_1 = 0.25, e_1)$ and $(1/\lambda_2 = 1.0, e_2)$. As illustrated by the covariance ellipse, using H^{-1} as covariance, the sampling scheme samples more in directions where the function increases the least and less in directions where the objective increases the most, which is important when combined with optimization.

4.1.2 Static problem In the previous example, we only analyzed the Hessian in a simple setting. Here, we consider the interaction between the local gradient and the geometry defined by the Hessian, and examine the sampling scheme in a more general setting.

We consider an optimization problem with an objective function $f_0(x)$ with $x \in \mathbb{R}^n$ and solve it with Newton's method by iteratively applying quadratic approximation and solving the subproblem. This process is fundamentally aligned with DDP. The subproblem with the approximation is given by

$$\min_{\delta x} \delta x^T \nabla f_0(\bar{x}) + \frac{1}{2} \delta x^T H \delta x, \text{ with } H = \nabla_{xx} f_0(\bar{x}),$$

where $\bar{x}$ is the current point, and we assume that the Hessian H is PD. Although a gradient-based step ($\delta x^* = H^{-1} \nabla f_0(\bar{x})$) reduces the cost of the quadratic model of the term $\delta x^T \nabla f_0(\bar{x})$, the PD Hessian serves as a local metric of cost sensitivity. It characterizes the sensitivity of the objective to deviations and how much step the optimizer can take. For example, when the Hessian has a large eigenvalue, the quadratic term ($\delta x^T H \delta x$) provides a large (positive)

penalty that may counteract the descent term from the gradient.

To facilitate exploration, we consider the situation where we perturb the solution by $\delta x + \xi$, where $\xi \sim \mathcal{N}(0, H^{-1})$ once in a few iterations as we do in ME-DDP. As we examined in the previous example, the eigenvectors of the Hessian indicate the principal axes of curvature. When the inverse Hessian is used as the covariance matrix, the sampling scheme becomes anisotropic: it prioritizes exploration along the axis of minimal sensitivity, where the quadratic model is flat and therefore safe to explore. Furthermore, it restricts sampling along the axis of maximal sensitivity, where even small perturbations would drive the cost up, making exploration unsafe. Here, we use the term “safety” to refer to the preservation of the optimization signal. When exploration imposes too high a cost, it essentially washes out the information of the gradient, which makes the update pure random sampling. The choice of covariance allows the optimizer to exploit information from the gradient without being perturbed by overly costly samples. Therefore, the choice provides a rigorous framework for broad exploration while preserving the efficiency of the optimization via gradient.

Another interpretation can be obtained by investigating the change of cost induced by sampling. We consider the perturbation $\delta x + \xi$, with noise $\xi \sim \mathcal{N}(0, \Sigma)$, where we specify the covariance Σ , later. By substituting the perturbed δx in the quadratic approximation and taking expectation of the stochastic components with ξ , we obtain:

$$\begin{aligned} \mathbb{E} \left[\frac{1}{2} \xi^T H \xi + \xi^T (\nabla f(\bar{x}) + H \delta x) \right] &= \frac{1}{2} \mathbb{E} [\text{Tr}(H \xi \xi^T)] \\ &= \text{Tr}(H \mathbb{E}[\xi \xi^T]) / 2 \\ &= \text{Tr}(H \Sigma) / 2 \\ &= \tau n / 2, \end{aligned}$$

where we take the covariance as a scaled inverse Hessian $\Sigma = \tau H^{-1}$ and use the fact that the trace of a scalar is the scalar itself. Generally, stochastic exploration imposes a penalty on the objective, particularly in regions of high curvature (large eigenvalue), where small perturbations can cause significant spikes. By this choice of covariance, we cancel the local geometry of the cost landscape. The optimizer explores more in the safe region and less in the unsafe region. This ensures that the expected cost of exploration remains constant and independent of the Hessian's eigenvalues, effectively normalizing the risk of sampling high-cost regions.

4.1.3 Regularization and covariance In the previous example, we assume that the Hessian is PD. In a practical setting, however, the Hessian may become indefinite or negative definite. To illustrate this, let us consider the two-dimensional example with three local minima whose surface plot is shown in Fig. 4 (a).

As illustrated in Fig. 4 (b), when the function is locally convex, the quadratic approximation for Newton's method is obtained with the original Hessian. Observe that the approximation yields a convex surface. In the same figure, we also perform a quadratic approximation at a locally concave point, where the Hessian is negative definite. At this point, we regularize the Hessian to obtain a valid quadratic

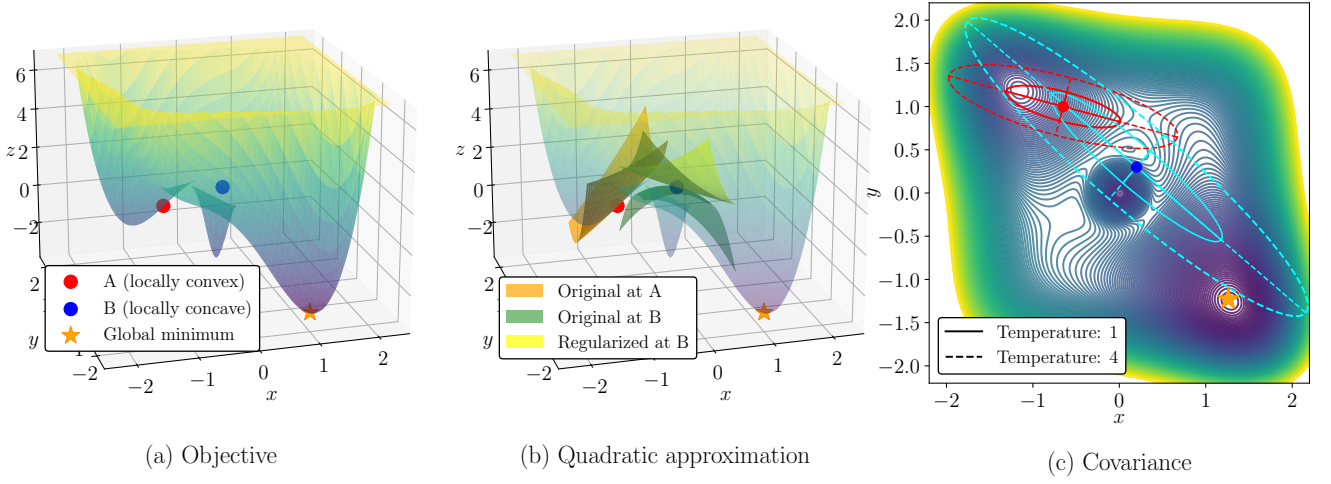


Figure 4. Surface plot of the objective function (a). Quadratic approximation of the objective at a locally convex point A and the approximation at a locally concave point B with original (not PD) and regularized (PD) Hessian (b). A contour plot of the objective function and the sampling covariance obtained at the two points A and B (c). The covariance ellipses, scaled by two temperatures, are drawn.

model for minimization. Because regularization preserves the original eigenvectors and shifts the eigenvalues, it maps the most negative eigenvalues of the original Hessian to the smallest positive eigenvalues of the regularized PD surrogate. Consequently, the resulting inverse covariance prioritizes exploration along the directions of the steepest descent. This ensures that sampling is performed more extensively along the primary axes of the manifold, facilitating escape from saddle points, local maxima, and landscapes similar to them.

Fig. 4 (c) illustrates the covariance ellipsoids induced by the approximations in Fig. 4 (b). To demonstrate the effect of temperature as a scaling factor, covariances at two different temperatures are projected onto the objective’s contour plot. The higher temperature corresponds to broader exploration, and therefore, the corresponding covariance is large.

4.1.4 Regularization with PD Hessian In **Background** section, we presented the importance of the regularization in DDP, however, with the first-order approximation of the dynamics and a proper choice of the cost function, i.e., PD Hessian with respect to state and control, Q_{uu} is theoretically guaranteed to be PD (see Appendix for detailed analysis).

Even in such cases, regularization is still triggered to achieve cost reduction or to make Q_{uu} well-conditioned for computing gains (Tassa et al. 2012). In these cases, since Q_{uu} is originally PD, covariance samples within a tighter region around the nominal trajectory effectively serve as an implicit trust-region mechanism. This mechanism ensures that exploration remains within the vicinity where the approximations are valid.

4.1.5 Inverse Hessian and Repulsive Force One can observe that the underlying principle of how the Hessian influences the repulsive force in SVN and, by extension, SV-DDP is the same as the inverse Hessian acting as a covariance matrix.

When solving (21), the repulsive force term is scaled by the inverse of the sum of Hessians of the Q function and that of the kernel. Consequently, the repulsive force is amplified in directions where the joint curvature is small. This ensures

that the rescaling does not impede the primary optimization objective, nor does it obstruct the spreading of trajectories. Recalling that the kernel value is highest when trajectories are in close proximity, this scaling mechanism steers the particles into low-sensitivity subspaces, facilitating efficient exploration.

4.2 Constrained handling via Relaxed log-barrier Function

This subsection details the integration of constraints into the ME-DDP framework (DDP and sampling). We begin by reviewing the standard log-barrier approach and its application in optimal control, followed by the introduction of the relaxed log-barrier formulation. Finally, we discuss the limitations of the method.

One common approach to incorporating constraints into an objective function is through the log-barrier function (Fiacco and McCormick 1968). These functions are powerful tools for static optimization and have been widely employed Murray and Wright (1994); Cho (2005); O’Neill and Wright (2020). The function is also used in the field of optimal control, specifically, in trajectory optimization (Jr. and Ho 1975; Betts 2001) and MPC (Wills and Heath 2004) under constraints.

A key limitation of standard log-barrier methods is that they cannot handle infeasible trajectories due to the domain restrictions of the log function. This is problematic in our setting because the exploration is only implicitly aware of constraints. To address this issue, we utilize relaxed-log barrier function (Hauser and Saccon 2006). We define the scalar relaxed barrier $\mathcal{B}_{r,i}(g_i; \mu, \delta_\mu)$ for a single constraint $g_i(x) < 0$ as

$$\mathcal{B}_{r,i}(g_i(x); \mu) = \begin{cases} -\mu \log(-g_i(x)), & g_i(x) \leq -\delta_\mu \\ \mu \left[\frac{1}{2} \left(\left(\frac{g_i(x) + 2\delta_\mu}{\delta_\mu} \right)^2 - 1 \right) - \log \delta_\mu \right], & g_i(x) > -\delta_\mu. \end{cases} \quad (25)$$

When $g(x) \in \mathbb{R}^w$, the total penalty is then given by the sum $\mathcal{B}_r(g(x)) = \sum_{i=1}^w \mathcal{B}_{r,i}(g_i(x))$. In this formulation, the function smoothly transitions from a logarithmic form to a polynomial approximation while maintaining C^2 continuity, allowing for infeasible trajectories.

The relaxed formulation has found applications in MPC and robotics. For instance, Feller and Ebenbauer (2017a,b) applies it to a linear MPC problem and provides a detailed performance analysis. In Aguiar et al. (2017), the function is incorporated into a projection-based motion planning algorithm Hauser (2002), achieving constrained trajectory optimization. The efficacy of the formulation with DDP/LQR is validated through hardware experiments on a quadruped in Grandia et al. (2019). Since it can handle infeasible trajectories, the formulation can be used that requires exploration that may cause constraint violation, such as reinforcement learning as proposed in Zhang et al. (2025).

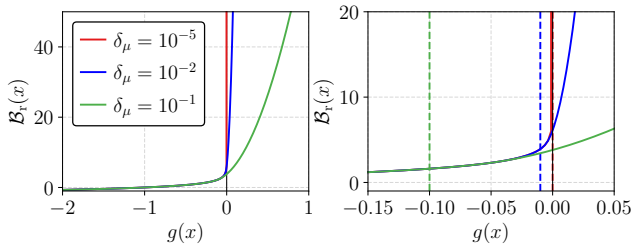


Figure 5. Relaxed log-barrier function $\mathcal{B}_r(x)$ for various switching points δ_μ . The right panel provides a magnified view near the origin to highlight the behavior around the boundary ($g(x) = 0$). Dashed vertical lines indicate the respective switching points δ_μ where the barrier transitions from a logarithmic to a quadratic form.

4.2.1 Properties of the function Fig. 5 provides the function with different relaxation parameters δ_μ . The parameter governs how closely the relaxed barrier approximates the exact logarithmic barrier and, consequently, the degree of constraint satisfaction. As $\delta_\mu \rightarrow 0$, the relaxed formulation converges to the exact log-barrier, yielding increasingly strict constraint satisfaction. However, this increased accuracy comes at the cost of numerical conditioning: steep gradients and large curvature arise near the constraint boundary, leading to ill-conditioned gradient and Hessian, which is critical in second-order optimization methods (Sleiman et al. 2021). Consequently, excessively small values of δ_μ recover the numerical instabilities of the exact log barrier. In practice, δ_μ is chosen to balance constraint satisfaction and numerical robustness (Feller and Ebenbauer 2017a). We note that even with a small δ_μ , the resulting conditioning issues are mitigated by appropriate scaling and regularization of gradient and Hessian terms (Nocedal and Wright 2006; Wächter and Biegler 2006).

In the original formulation for static problems, the penalty parameter μ is reduced towards zero as optimization proceeds. In practice, however, a small fixed μ often provides sufficient accuracy while improving numerical stability (Grandia et al. 2019). We present how the barrier function is incorporated into DDP in appendix.

4.2.2 Limitations Numerical instability can be mitigated by the method described above, but numerical sensitivity,

i.e., the amplification of small differences near the constraint boundary and hard branching, cannot be fully removed. We provide a detailed analysis in the appendix. We consider this to be an inherent limitation of the barrier approach compared with other constrained optimization methods, such as the Augmented Lagrangian (AL) method (Hestenes 1969; Powell 1969; Rockafellar 1973), which incorporates Lagrangian multipliers for constraint satisfaction. The use of multipliers enables the algorithm to satisfy constraints and optimality conditions without requiring excessively large penalty parameters, which are a known source of numerical ill-conditioning and instability (Nocedal and Wright 2006). Nevertheless, we employ the barrier function because it can rapidly divert trajectories away from infeasible regions during the early stages of optimization.

4.3 Receding Horizon Implementation

To meet real-time requirements in highly nonlinear environments, as we will see in our experiments, we adopt an MPC formulation. A critical observation in our implementation is the necessity of consecutive optimization iterations for constraint satisfaction. Because the exploration process is not explicitly aware of the constraints, raw samples often violate constraints. We observe that overly frequent sampling is counterproductive, as it disrupts the optimizer’s ability to recover feasibility. By restricting exploration to the initiation of the MPC cycle and prioritizing successive DDP iterations, we ensure the control sequence is both diverse and respects constraints.

A challenge in warm-starting is the coupling between the sampling covariance $\Sigma(Q_{uu}^{-1})$ and the nominal trajectories. While the control sequence $U^{(n)}$ can be shifted for warm-starting, the state sequence $X^{(n)}$ obtained by shifting inevitably diverges from those obtained by applying $U^{(n)}$ to the system with initial state x_t . Because feedback gains K and sampling covariances Σ are local approximations, this mismatch in state trajectory yields the simple temporal shift of feedback gain and covariance invalid. Consequently, a re-evaluation of the system’s sensitivity via a backward pass is necessary at each time step to ensure the exploration remains aligned with the current local geometry.

We perform an initial backward pass at the start of each MPC loop to generate feedback gain and covariance. When the backward pass fails due to an ill-conditioned Q_{uu} , the algorithm samples with a pre-specified fixed covariance used in the initialization.

5 Experiments

In this section, we provide two experimental results.

First, we conduct extensive simulations across four robotic systems, comparing seven algorithms, including DDP, ME-DDP variants, and MPPI variants. Second, we present a hardware experiment with a quadrotor to validate the performance and real-time feasibility of the ME-DDP variants.

5.1 Numerical Experiments

The purpose of the experiment is to observe the properties of gradient-based (DDP), sampling-based (MPPI), and hybrid

(ME-DDP) algorithms. We have comparison in performance and computational efficiency.

5.1.1 Dynamics and Tasks We tested four dynamics: 2D car, quadrotor, Ant, and Barkour (quadruped) (Caluwaerts et al. 2023). The Ant and Barkour are simulated in the Jax-based physics engine Brax (Freeman et al. 2021). The task for each system is to reach targets while navigating obstacle fields. We use circular (2D) or cylindrical (3D) obstacles, generated in a pseudo-random fashion. To ensure rigorous testing, we manually introduce additional obstacles in certain configurations to create challenging non-convex landscapes, including local-minima traps. Here, we provide brief descriptions of the systems. Ant and Barkour are shown in Fig. 6.

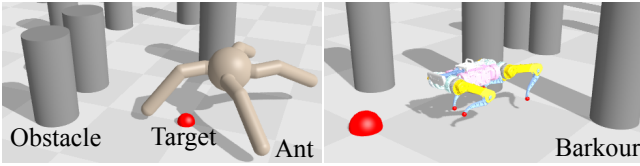


Figure 6. Experimental overview of Ant and Barkour. The task here is to reach the target while avoiding cylindrical obstacles.

2D car: The state $x \in \mathbb{R}^3$ comprises 2D positions and heading angle. The control $u \in \mathbb{R}^2$ consists of transitional and angular velocities. The discretization interval is 0.02 s. To investigate the effect of the MPC look-ahead horizon, we tested short (50 steps) and long (70 steps) horizons.

Quadrotor: The state $x \in \mathbb{R}^{12}$ includes 3D positions, Euler angles, and their time derivatives. The control $u \in \mathbb{R}^4$ is the thrust force generated by four rotors. We follow the dynamics established in Luukkonen (2011). The discretization interval is 0.01 s. We have short (50 steps) and long (70 steps) horizons as in the case of the 2D car.

Ant: The Ant has four legs, with two joints. The state $x \in \mathbb{R}^{29}$ contains the 3D position and orientation of the body in the quaternion, the angles of the legs, and their time derivatives. The control $u \in \mathbb{R}^8$ is torque applied to the joints. The control loop runs at 0.01 s with a simulation step of 0.002 s, integrating 5 substeps per control cycle.

Barkour: The quadruped has state $x \in \mathbb{R}^{37}$ comprising the 3D position, the body quaternion, and the three joint angles for each of the four legs, including the abductors, hips, and knees. The control $u \in \mathbb{R}^{12}$ is the command for the joint angles of the legs relative to the default angles that correspond to the standing posture. The control loop runs at 0.02 s with a simulation step of 0.006 s, integrating 3 substeps per control cycle.

We note that despite the higher dimensionality of the quadruped compared to the Ant, the use of a position-based (or angle-based) control interface provides a more structured search space, especially for MPPI. Position-level actions effectively abstract away the high-frequency dynamics and stability issues of torque-level control. This allows MPPI to discover task-relevant trajectories more efficiently. Typically, with torque-based control, many trajectories are wasted because random torque cannot even let robots stand stably. With position-based control, the optimizer can search for candidate trajectories within quasi-static equilibrium poses;

therefore, the search space is significantly reduced to a region of low-cost configurations.

5.1.2 Algorithms We compare seven algorithms across three families: normal DDP, three ME-DDP variants, and three MPPI variants. For the Ant and Barkour tasks, the DDP-based algorithms are modified to operate on the manifold S^3 to handle quaternions. We project quaternion deviations into the tangent space. This ensures that the quadratic approximations remain valid while the unit-norm constraint is maintained via retraction back to the manifold (Boutsellis and Theodorou 2021).

For MPPIs, we have three different policy parameterizations and update rules. They are Unimodal Gaussian MPPI (UG-MPPI), Multimodal Gaussian MPPI (MG-MPPI), and UG policy with SV update (SV-MPPI) (Wang et al. 2021). Here, the policy parameterizations correspond to UG-, MG-ME-DDP, and SV-DDP. The experiment is performed in MPC fashion.

A key distinction between these DDP and MPPI families lies in their mechanism for constraint satisfaction. Due to the requirement of differentiability, DDP-based methods incorporate constraints into the cost function using barrier functions explained in section **Essential Components in SV-DDP**. While this allows for gradient-based optimization, it can permit the algorithm to violate constraints. In contrast, MPPI utilizes indicator functions with a prohibitive cost to encode constraints. The control update in MPPI is a weighted average of the sampled control trajectories based on their associated costs. Consequently, infeasible trajectories are assigned negligible weights. This mechanism effectively excludes unsafe trajectories from the control computation. Although it does not theoretically guarantee constraint satisfaction, our experiments show that it works well in practice to maintain feasibility given that MPPIs find a path to the targets.

5.1.3 Parameter Tuning and Optimization The performance of both ME-DDP and MPPI is affected by hyperparameter selection, such as the sampling covariance in MPPI or the temperature parameter τ in ME-DDP. Although the sampling covariance in the ME-DDP variants arises naturally from the inverse Hessian of the Q function, the temperature must still be tuned to balance exploration and exploitation. To ensure a fair comparison, we acknowledge that different algorithms may require different cost weightings to achieve peak performance, even when the underlying structure (e.g., quadratic in position error to goal) is shared.

Consequently, we utilize a Bayesian hyperparameter optimization framework, specifically the Tree-structured Parzen Estimator (TPE), to tune both the policy parameters and the cost weights for every algorithm using an open-source Neural Network Intelligence (NNI) toolkit (Microsoft 2021).

For the tuning process, we define a high-level objective that prioritizes early entry into the target region and remaining within it for a prescribed duration, while penalizing the failure to reach the target within the specified time steps.

We use a fixed number of modes/trajectories $N = 8$ in ME-DDP variants. The samples/trajectories of MPPIs vary with the complexity of the system. For multimodal policies

of MPPI, i.e., MG- and SV-MPPI, we use the same number of modes $N = 8$ and distribute an equal number of particles per mode.

2D Car and Quadrotor: We utilize two sparse and two dense obstacle environments for tuning. This diversity is necessary because algorithms tuned solely on simple environments tend to converge toward low temperatures or narrow sampling covariances, which fail to generalize to complex, non-convex landscapes. The number of samples used in MPPI is 2048 for the 2D car and 8000 for the quadrotor.

Ant: Due to the higher computational cost of these high-dimensional systems, we utilize a single representative environment for the tuning process. The number of samples in MPPI is 8192.

Barkour: As in the case of Ant, we use one environment for tuning, and the number of samples in MPPI is 8192.

A primary challenge in tuning the cost function for the quadruped Barkour is to balance its performance with the requirement for gait realism. When the yaw rotation has less penalty compared to the position, the quadruped rotates while avoiding obstacles and moving forward. Although it can successfully reach the target without colliding, we prefer a realistic gait to spinning as provided in Fig. 7.

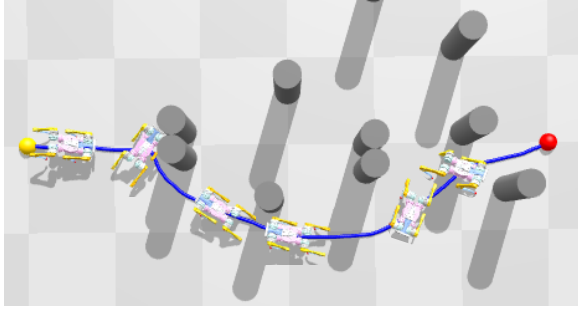


Figure 7. Impact of independent coordinate tuning on Barkour. The absence of yaw-position coupling leads to excessive angular excursions and rotational instability, despite the optimizer, UG-MPPI, successfully hitting the target.

To mitigate the issue, we introduce coupled weights of position and yaw during the parameter search. This coupling ensures that when the tuner commands a high penalty on the position, the angle is also penalized to prevent spinning.

In our experiments, we observed a divergence in how optimizers responded to this coupling. While the coupling significantly improved the qualitative trajectory quality (realistic gait) of the DDP-based solvers, it led to a degeneration in the success rate of MPPIs. Consequently, while MPPI can achieve comparable or even higher performance in unconstrained scenarios, it produces physically controversial behaviors that are less suitable for realistic hardware deployment than DDPs.

5.1.4 Results To capture the stochastic nature of the algorithms, we run ten trials per environment except for the normal deterministic DDP. We have 16 environments for 2D car and quadrotor (comprising eight sparse and eight dense configurations), and ten environments for Ant and Barkour. This resulted in a total of 160 trials for the car and quadrotor experiments and 100 trials for the Ant and

Barkour, from which we provide statistics. The overviews of the experiments are shown in Fig. 8 for the 2D car and quadrotor, Fig. 9 for Ant, and Fig. 10 for Barkour.

Performance: Table. 1 shows the statistics of the results. We have two categories of results based on the constraint violation. The first group is feasible trajectories. This group includes trials where the system successfully reached the goal region (defined by an L_2 norm less than 0.3) while maintaining a maximum obstacle constraint violation of less than 10^{-6} . For this group, we report the Success rate [%], the mean time step to steady state denoted as Time. This metric is defined as the first time step the agent enters and remains in the goal region for at least 10 consecutive steps. Finally, we report the mean path length denoted as Path. The second group relaxes the requirement of constraint satisfaction. This group represents a broader set of trials that reached the goal but allowed for constraint violations. In addition to the three metrics mentioned above, we provide the average constraint violation (Violation). We have the movies of the 2D car in extension 1, Quadrotor in extension 2, Ant in extension 3, and Barkour in extension 4. For Barkour, we also provide the results for MPPIs without position-yaw coupling as described in the **Parameter Tuning and Optimization** section.

The ME-DDP variants consistently outperform standard deterministic DDP, demonstrating that the exploration mechanism helps optimizers explore and find better local minima.

In low-dimensional systems (e.g., 2D car, quadrotor), gradient-based methods exhibit a superior success rate and efficiency through shorter time and path length. Our mechanism using local geometric information, e.g., the gradient and Hessian for optimization and the Hessian for sampling covariance, outperforms purely sampling methods. This suggests that for these systems, local geometric information, specifically the gradient and Hessian, is sufficient to characterize the cost landscape.

In high-dimensional locomotion, the cost landscape is characterized by numerous local minima, making the role of exploration critical. The results demonstrate that while ME-DDP achieves a higher overall success rate, MPPI is capable of steering the system more aggressively, as evidenced by shorter completion times and path lengths. This suggests that while gradient-based methods like ME-DDP often converge to the local basins of their nominal trajectories, MPPIs can discover solutions that DDPs cannot find, such as the yaw-rotation in Barkour (Fig. 7) or the jumping gait in the Ant environment presented in extension 3. Essentially, MPPI can potentially find superior, high-energy modes that DDP cannot, at the cost of a higher failure rate due to the lack of gradient-guided refinement.

Although multimodal policies are designed to capture diverse paths, the results suggest that their efficacy depends heavily on the degree of mode separation and the quality of representative trajectories. In our experiments, we observed that an unimodal policy can sometimes outperform other policies in both DDPs and MPPIs. This is because when the policy fails to separate the trajectories into distinct, meaningful classes, it has a lower local sampling density than the unimodal policy. Furthermore, it fails to provide a broader exploration capability. Consequently, they cannot be as efficient as a focused unimodal policy. While it

Table 1. Benchmark results comparing optimization-based ME-DDP variants and sampling-based MPPI baselines. The results are categorized into feasible and all (including infeasible) trajectories. Success rate shows the percentage of the trajectories that reach the goal. Time is the average first time step when the trajectory enter the goal region and stays there for ten time steps. Path is the average trajectory length. Violation is constraint violation computed over infeasible trajectories. The arrow $\uparrow$ indicates a higher value is preferred, while $\downarrow$ indicates a lower value is preferred.

Dynamics	Algorithm	Feasible Trajectories			All (Including Infeasible Trajectories)			
		Success rate [%] $\uparrow$	Time $\downarrow$	Path $\downarrow$	Success rate [%] $\uparrow$	Time $\downarrow$	Path $\downarrow$	Violation $\downarrow$
2D Car $n_x \in \mathbb{R}^4$ $n_u \in \mathbb{R}^2$ $T_{\text{mpc}} = 50$	DDP	40.0	118	11.6	40.0	118	11.6	✓
	UG-ME-DDP	66.3	149	12.4	75.6	153	12.6	1.96×10^{-4}
	MG-ME-DDP	81.2	201	12.5	89.4	201	12.5	2.16×10^{-4}
	SV-DDP	81.9	136	12.5	93.8	138	12.6	2.44×10^{-4}
	UG-MPPI	50.0	157	13.6	50.0	157	13.60	✓
	MG-MPPI	45.6	156	13.6	45.6	156	13.56	✓
	SV-MPPI	78.1	175	13.7	78.1	175	13.65	✓
2D Car $n_x \in \mathbb{R}^3$ $n_u \in \mathbb{R}^2$ $T_{\text{mpc}} = 70$	DDP	31.3	127	12.4	31.3	127	12.4	✓
	UG-ME-DDP	75.6	142	13.2	80.0	143	13.3	4.74×10^{-4}
	MG-ME-DDP	74.4	145	13.5	81.3	147	13.6	1.56×10^{-4}
	SV-DDP	85.0	135	12.8	95.0	138	12.9	2.68×10^{-4}
	UG-MPPI	62.5	151	12.9	62.5	151	12.86	✓
	MG-MPPI	63.1	162	13.6	63.1	162	13.62	✓
	SV-MPPI	82.5	153	13.2	82.5	153	13.21	✓
Quadrotor $n_x \in \mathbb{R}^{12}$ $n_u \in \mathbb{R}^4$ $T_{\text{mpc}} = 50$	DDP	37.5	145	9.55	43.8	155	10.2	5.28×10^{-6}
	UG-ME-DDP	77.5	140	9.20	83.1	141	9.26	1.32×10^{-2}
	MG-ME-DDP	76.9	139	9.00	80.0	141	9.05	1.10×10^{-4}
	SV-DDP	83.1	132	8.91	89.4	132	8.89	1.19×10^{-4}
	UG-MPPI	43.8	137.7	8.92	43.8	138	8.92	✓
	MG-MPPI	62.5	155.0	9.16	62.5	155	9.16	✓
	SV-MPPI	75.0	152.7	9.14	75.0	153	9.14	✓
Quadrotor $n_x \in \mathbb{R}^{12}$ $n_u \in \mathbb{R}^4$ $T_{\text{mpc}} = 70$	DDP	3.13	189	11.0	5.00	193	11.1	7.71×10^{-2}
	UG-ME-DDP	91.3	141	9.36	95.6	141	9.32	4.20×10^{-5}
	MG-ME-DDP	83.8	153	9.20	85.0	153	9.19	1.75×10^{-4}
	SV-DDP	87.5	136	8.91	94.4	136	8.88	7.66×10^{-5}
	UG-MPPI	69.4	148.2	9.17	69.4	148	9.17	✓
	MG-MPPI	69.4	143.9	9.18	69.4	144	9.18	✓
	SV-MPPI	67.5	186.4	9.39	67.5	186	9.39	✓
Ant $n_x \in \mathbb{R}^{29}$ $n_u \in \mathbb{R}^8$ $T_{\text{mpc}} = 40$	DDP	20.0	174	10.8	20.0	174	10.8	✓
	UG-ME-DDP	55.0	134	11.1	57.0	138	11.1	6.49×10^{-2}
	MG-ME-DDP	82.0	112	11.0	83.0	113	10.6	2.50×10^{-4}
	SV-DDP	80.0	119	11.5	80.0	119	11.5	✓
	UG-MPPI	53.0	73.2	9.79	53.0	73.2	9.79	✓
	MG-MPPI	54.0	65.6	9.34	54.0	65.6	9.34	✓
	SV-MPPI	48.0	66.0	9.39	48.0	66.0	9.39	✓
Barkour $n_x \in \mathbb{R}^{37}$ $n_u \in \mathbb{R}^{12}$ $T_{\text{mpc}} = 50$	DDP	10.0	448	6.02	30.0	458	6.40	1.21×10^{-4}
	UG-ME-DDP	80.0	311	6.24	81.0	311	6.25	9.13×10^{-6}
	MG-ME-DDP	60.0	282	6.36	76.0	288	6.38	1.56×10^{-4}
	SV-DDP	91.0	323	6.31	92.0	324	6.32	7.95×10^{-5}
	UG-MPPI	35.0	291	6.14	35.0	391	6.14	✓
	MG-MPPI	30.0	245	6.08	30.0	245	6.08	✓
	SV-MPPI	43.0	343	6.29	43.0	334	6.29	✓
Barkour-Decouple-Position-Yaw^a	UG-MPPI	95.0	237	6.29	95.0	237	6.29	✓
	MG-MPPI	58.0	250	6.26	58.0	250	6.26	✓
	SV-MPPI	71.0	276	6.25	71.0	276	6.25	✓

^a Decouple position and yaw weights of the cost during tuning.

remains challenging to draw a universal conclusion on which algorithm consistently outperforms the others across all domains, SV-DDP demonstrates remarkable reliability,

consistently maintaining a success rate equal to or greater than 80 %.

Computational Time: We compare computational performance of the DDP, ME-DDP variants, and UG-MPPI using

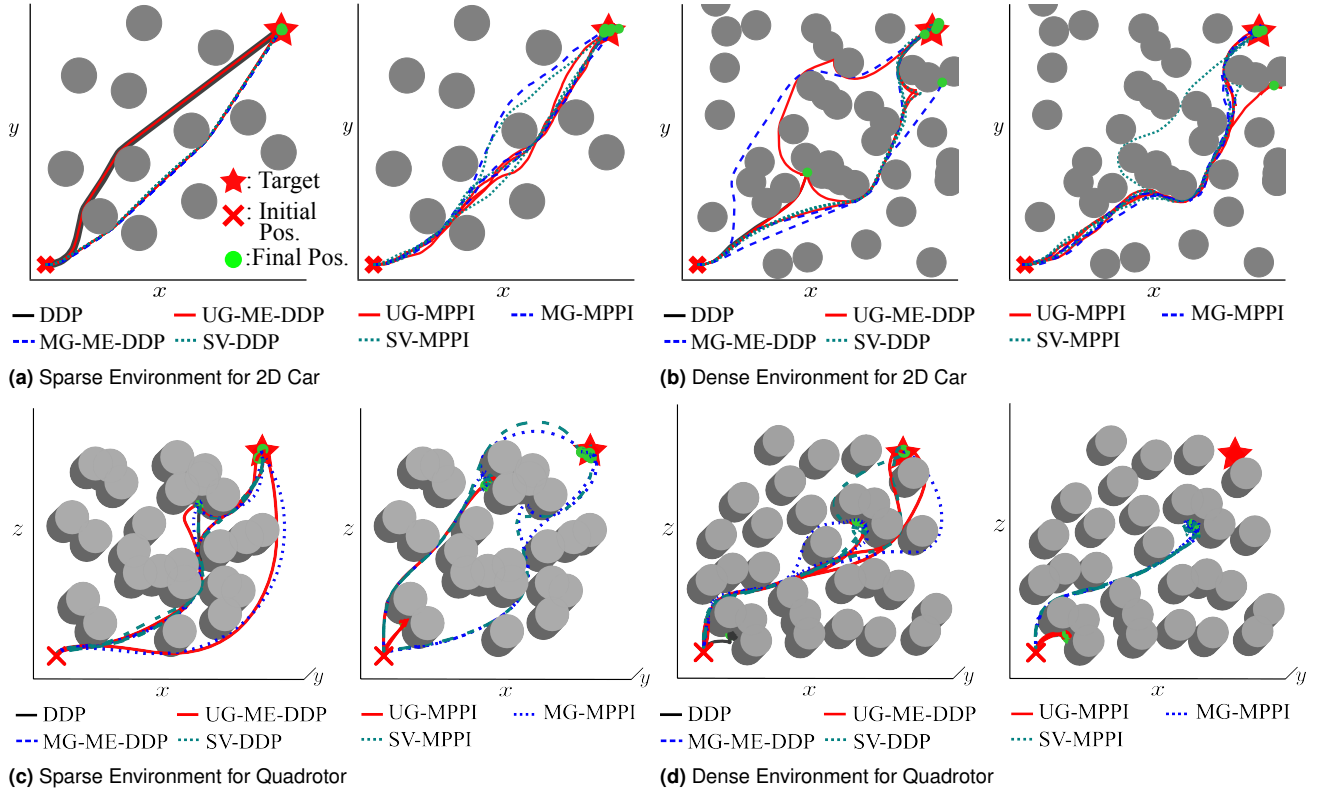


Figure 8. Trajectory comparisons for 2D Car and Quadrotor environments. Within each panel, the left image shows DDP, and the right shows MPPI.

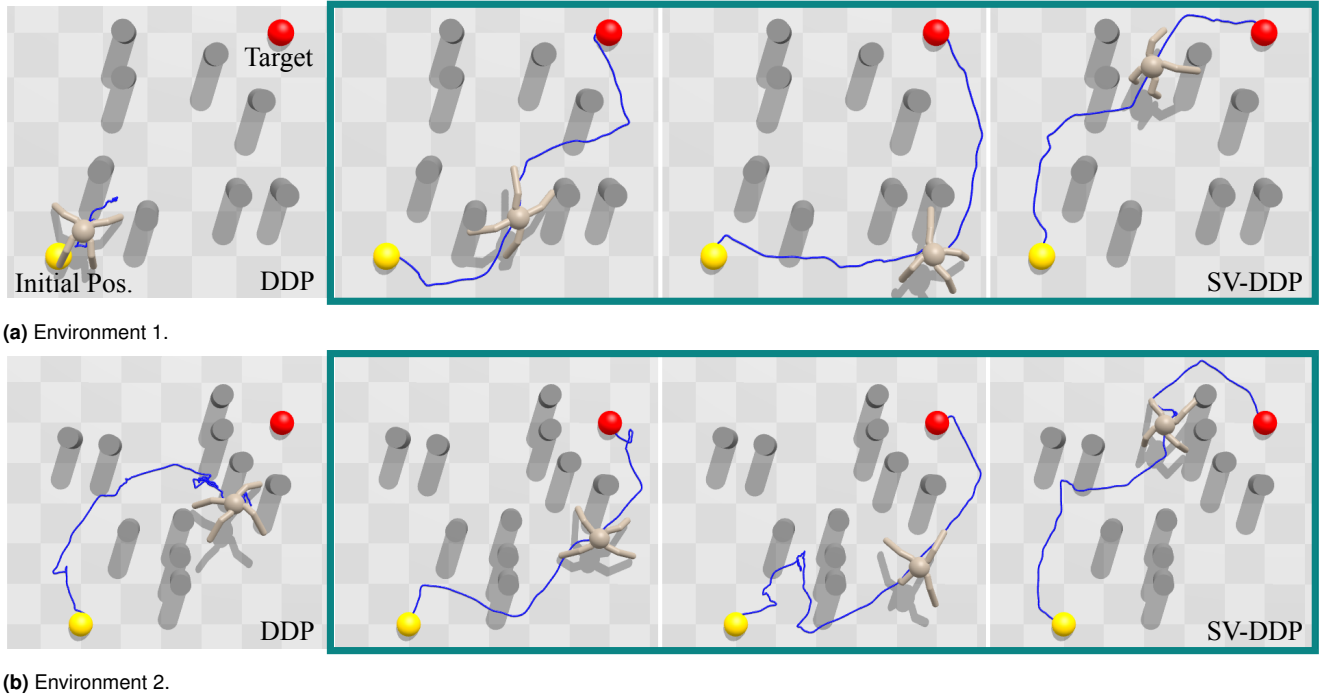


Figure 9. Result of Ant experiments. Trajectories get stuck at poor local minima, and three distinctive trajectories that successfully reach the target via SV-DDP are presented.

both 2D car and quadrotor dynamics across two look-ahead horizons ($T_{\text{mpc}} = 50$ and $T_{\text{mpc}} = 70$). Both algorithms are implemented in JAX [Bradbury et al. \(2018\)](#). Statistics were computed over 200 MPC time steps. As shown in Table 2, MPPI exhibits significantly faster computational speed compared to DDP-based methods. This is because MPPI

bypasses the computation of the gradient and Hessian that are required for DDPs during their backward pass. Although the exploration mechanisms in ME-DDP variants introduce additional computational overhead to the normal deterministic DDP, they still meet real-time requirements. Specifically, given the discretization intervals of 20 ms for the 2D car and

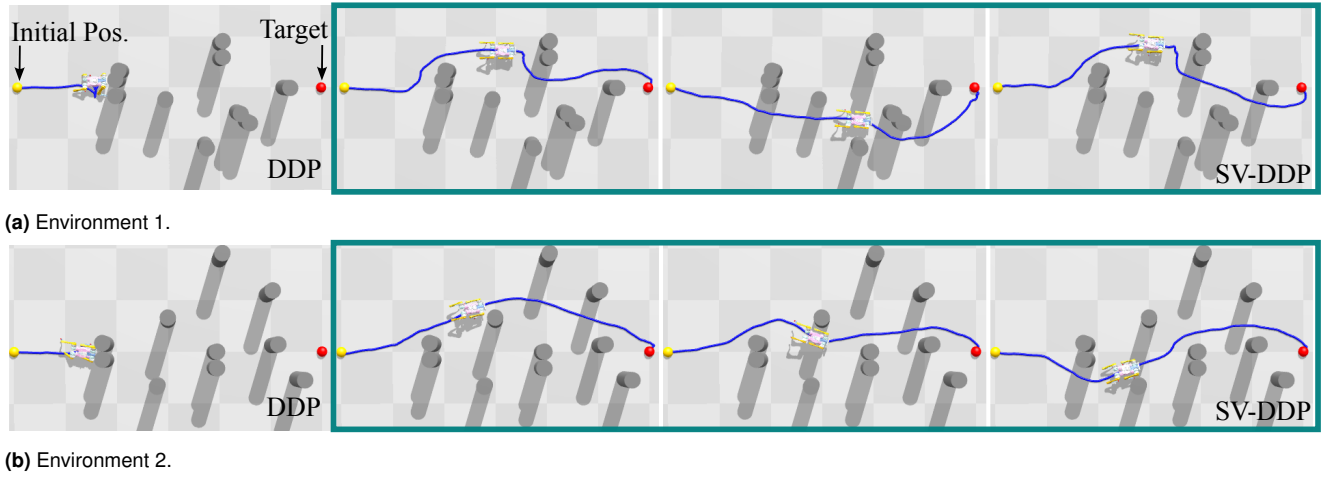


Figure 10. Result of Barkour experiments. Trajectories get stuck at poor local minima, and three distinctive trajectories that successfully reach the target via SV-DDP are presented.

Table 2. Computational performance benchmark. Comparison of mean execution time and standard deviation [ms] of one time step for the 2D car and quadrotor systems across two look-ahead horizons ($T_{MPC} = 50, 70$).

Algorithm	2D Car Time [ms]		Quadrotor Time [ms]	
	$T_{mpc} : 50$	$T_{mpc} : 70$	$T_{mpc} : 50$	$T_{mpc} : 70$
DDP	19.46 $\pm$ 3.06	25.70 $\pm$ 3.12	21.53 $\pm$ 2.24	29.51 $\pm$ 2.40
UG-ME-DDP	24.45 $\pm$ 1.54	32.39 $\pm$ 1.15	26.50 $\pm$ 1.40	35.73 $\pm$ 1.07
MG-ME-DDP	23.96 $\pm$ 1.10	32.25 $\pm$ 1.28	25.60 $\pm$ 0.95	35.87 $\pm$ 0.98
SV-DDP	25.53 $\pm$ 3.61	33.54 $\pm$ 3.78	26.54 $\pm$ 1.14	37.34 $\pm$ 1.48
MPPI	2.10 $\pm$ 0.31	2.22 $\pm$ 0.47	2.83 $\pm$ 0.65	3.91 $\pm$ 0.80

10 ms for the quadrotor, the optimization finishes before the next control cycle, ensuring the feasibility of the proposed approach for robotic control. Furthermore, the low execution variance observed across all methods indicates a highly deterministic timing profile, which is critical for maintaining stable control of systems.

For the computational benchmark, we focus on the 2D car and quadrotor to ensure that the timings reflect the algorithmic complexity of the optimizers rather than the overhead of the physics engine. Using simple dynamics, we isolate the problem of heavy computation of the propagation of dynamics and other steps in the optimization loop. By doing so, we can provide a clearer assessment of the real-time feasibility of the algorithms.

All timing comparison experiments were conducted on a system equipped with an Intel i9-13900K CPU, a 64Gb of system memory, and an NVIDIA RTX4090 GPU.

5.2 Hardware Experiments

To validate the real-time performance and robustness of the proposed ME-DDP framework, we conducted a series of hardware experiments using a quadrotor platform. While simulations provide a controlled environment for algorithmic comparison, physical hardware introduces real-world challenges such as sensor noise and aerodynamic turbulence. In this section, we provide experimental setup and results.

5.2.1 Hardware and Experimental Setup: The experimental platform is a custom-built quadrotor, as illustrated in

Fig. 11. The system architecture is divided into three primary components: sensing, computation, and flight control.

State Estimation: To provide global state estimation, we utilize a VICON motion capture system. The quadrotor's full state is estimated from VICON data using EKF of the PX4-based flight controller (Meier et al. 2015).

Ground Station and Optimization: A ground station receives the state telemetry via a Local Area Network. This station serves as the primary computational resource, computing the DDPs in MPC for trajectory optimization. The optimized control sequences are then sent back to the vehicle via a dedicated Wi-Fi link. The control frequency is set to 10 Hz.

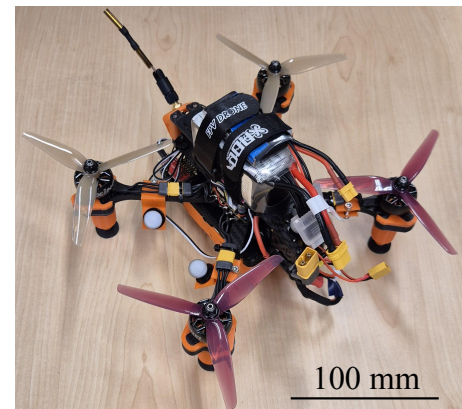


Figure 11. The quadrotor platform used for experimental validation.

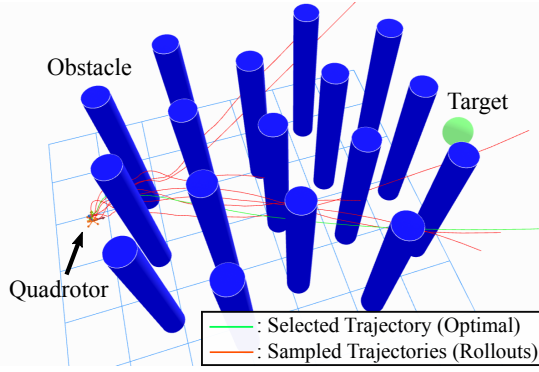


Figure 12. 3D visualization of a hardware experiment.

Onboard Systems: The vehicle is equipped with a Raspberry Pi onboard computer, which acts as the bridge between the ground station and the flight hardware. The onboard computer communicates with a flight controller. The controller runs position control based on the optimal state trajectory computed by DDPs. The controller translates the received control commands (positions) into low-level motor signals.

5.2.2 Results We test the performance of the DDP family, i.e., standard deterministic DDP and three ME variants (UG-ME-DDP, MG-ME-DDP, and SV-DDP), across two environments with different cylindrical obstacle configurations. An overview of the experiment is visualized in Fig. 12. To evaluate the robustness of each algorithm and characterize the stochastic behavior inherent in the ME-DDP variants, we conduct multiple trials for each algorithm across different environments.

Instead of physical obstacles, we used virtual representations. For qualitative evaluation, these virtual obstacles are overlaid on a top-down camera feed of the experimental arena. In [extension 5](#) and [extension 6](#), we present a synchronized side-by-side visualization, with the overlaid real-world video on one side and the corresponding ROS visualization on the other.

The results of the experiments are provided in Table. 3, and the trajectories of the quadrotor and overlaid obstacle are shown in Fig 13. Environment 1 shown in Fig 13 (a) is a forest of obstacles. As illustrated by the overlaid trajectories, without exploration, the algorithm finds a simple straight path, which acts as a detour to avoid the obstacles. With exploration, on the other hand, it actively explores the cost landscape and discovers shorter, more direct passages through the forest.

Environment 2 in Fig 13 (b) has some specific concave configurations designed to induce multiple poor local minima that can cause deadlocks. In addition to these, the environment incorporates forest-type configurations, as in environment 1. This environment tests the capability of the proposed sampling-based policies to escape these local minima. Furthermore, it simultaneously examines the exploration capability demonstrated in environment 1.

In Table 3, we classify each trial into four categories. First, a trial is classified as Safe Success, when the quadrotor reaches the target without violating constraints. This represents the ideal outcomes. Second, the category Goal with Coll. is for the cases where the quadrotor reaches the

target but hits obstacles. For these two categories, we provide the average time to reach the target. Finally, Failure to Goal represents cases where the quadrotor fails to hit the target, which we further divide into two groups based on the failure modes. Deadlock without Coll. occurs when the quadrotor is captured by a poor local minimum and spends too much time there. Coll. is for cases where the quadrotor can escape from the deadlock, but subsequently collides with an obstacle and times out. We distinguish between these two modes to provide a more granular analysis of constraint satisfaction.

Overall, the sampling mechanism facilitates exploration, and since the constraints are encoded as part of the cost, it also facilitates constraint satisfaction.

The hardware results confirm the trends observed in simulation, where the multimodal policy fails to provide better solutions compared to the standard unimodal one. We observed that without an explicit separation mechanism, the multimodal policy essentially becomes a unimodal one with fewer trajectories.

In this case, the multimodal policy has a lower local sampling density than the unimodal policy. Furthermore, it fails to provide a broader exploration capability. Consequently, it may act as a middle ground of poor performance: it is not as efficient as a focused unimodal policy, nor as diverse as a SV policy.

This phenomenon explains the higher deadlock rates observed for the MG-ME-DDP. The policy wasted its exploration budget without successfully identifying alternative routes. In contrast, the unimodal UG-ME-DDP prioritizes a high-resolution search around the most viable candidate.

6 Conclusion

This work builds upon the Maximum Entropy (ME) framework to provide a unified perspective on the hybrid control mechanism that combines gradient- and sampling-based optimization.

While the core objectives of the ME framework have been previously established, we provide an analytical foundation that clarifies the role of the control Hessian Q_{uu} . Specifically, we provide a rigorous way of understanding the physical and mathematical interpretations of sampling from the inverse of Q_{uu} , revealing how this mechanism interacts with the non-convex cost landscapes inherent in robotics.

Our benchmarking across four systems in simulation reveals a distinct trade-off between structured optimization and stochastic exploration, advocating for a hybrid optimization approach that moves beyond pure sampling:

- 1. Low-Dimensional Systems (2D Car, Quadrotor):** In these systems, ME-DDP variants demonstrate the benefits of leveraging local information for optimization and exploration. We found that the local gradient and Hessian provided by the DDP backward pass achieve superior success rate, and more efficient path lengths compared to MPPIs that rely on pure sampling.
- 2. High-Dimensional Systems (Ant, Barkour):** In complex systems, ME-DDP remains highly reliable, maintaining a high overall success rate due to its

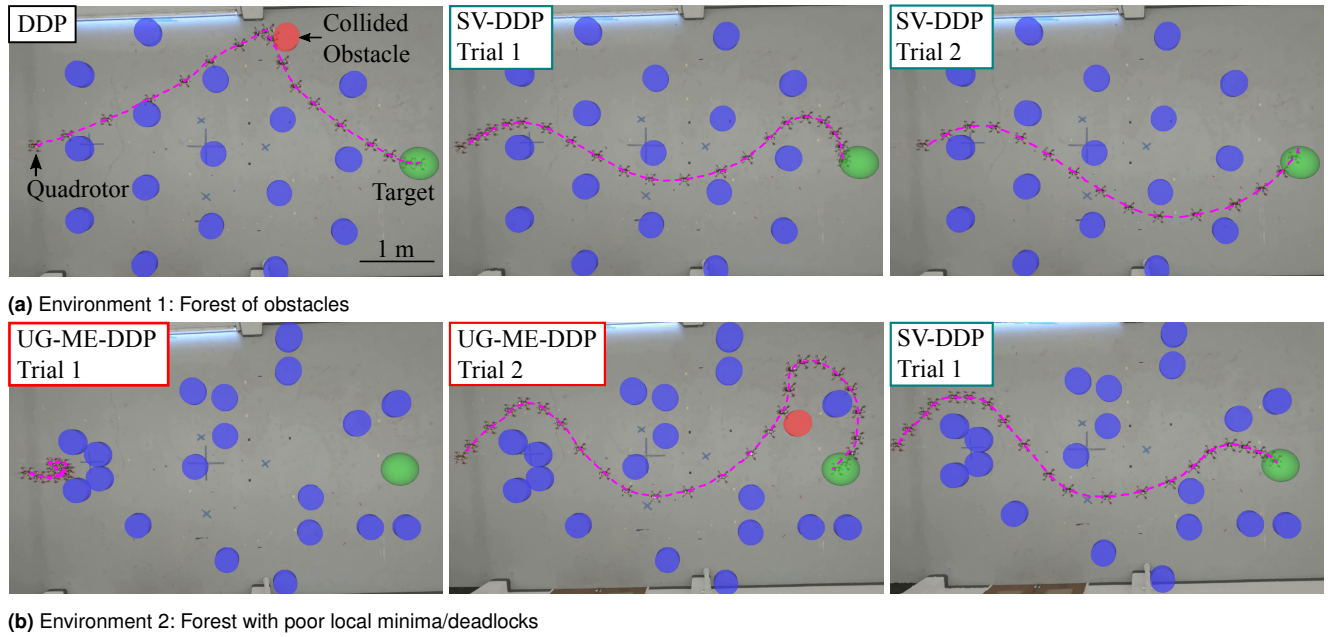


Figure 13. Representative resulting trajectories from hardware experiments in two different environments. The drone positions were captured at 0.5 s intervals. Obstacles are colored based on their states. Blue indicates a collision-free, while red denotes a collision verified by ground truth position. In the left panel of (b), the quadrotor is captured at a deadlock.

Table 3. Hardware Performance Analysis. Outcomes are categorized into successful runs and failures with collision. Failures are further distinguished by whether the quadrotor reached the target (Goal with Coll.) or spent too much time on a poor local solution and cannot reach the target with or without collision (Deadlock without Coll./Coll.). The sum of the four outcome categories for each method equals 100 %.

Algorithm	runs	Safe Success		Goal with Coll.		Failure to Goal	
		rate[%]	time[s]	rate[%]	time[s]	Deadlock without Coll.[%]	Coll.[%]
DDP	9	11.1	11.71	44.4	14.06	33.3	11.1
UG-ME-DDP	9	66.7	13.56	22.2	12.30	11.1	0.0
MG-ME-DDP	11	54.5	14.17	27.3	11.92	18.2	0.0
SV-DDP	12	83.3	13.37	8.3	11.05	8.3	0.0

structured refinement. However, we observed that MPPI is capable of discovering global modes that DDP-based methods may miss. While MPPI can struggle with success rates due to the lack of gradient-based refinement, its exploratory nature offers a unique advantage in discovering non-local solutions.

Regarding practical implementation, we demonstrate that while MPPI is computationally efficient per iteration, our DDP-based variants are sufficiently fast for real-time quadrotor control. We validate the efficacy of the algorithms through hardware experiments with a quadrotor navigating cluttered environments, proving that the proposed hybrid approach is robust for real-world deployment.

We conclude that for most robotics applications, providing a rigorous local structure through the ME-DDP framework offers a reliable path to high-performance control while remaining computationally feasible for hardware deployment.

Future research directions include implementing the full feedback structure of the optimal controller on hardware, exploring alternative mechanisms for constraint satisfaction beyond relaxed-log barrier functions, and establishing theoretical convergence guarantees and conditions.

Author Contributions

Yuichiro Aoyama: Conceptualization, Formal analysis, Investigation, Software, Methodology, Visualization, Writing-original draft. **Minchan Jung:** Conceptualization, Formal analysis, Investigation, Software, Methodology, Visualization, Validation, Writing-review & editing. **Akash Ratheesh:** Investigation, Software, Methodology, Visualization, Writing-review & editing. **Evangelos A. Theodorou:** Conceptualization, Supervision, Project administration, Writing-review & editing.

Statements and Declarations

Ethical considerations

This article does not contain any studies with human or animal participants.

Consent to participate

This article does not contain any studies with human or animal participants.

Consent for publication

Not applicable.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article:

Akash Ratheesh and Evangelos A. Theodorou were supported by the National Aeronautics and Space Administration under University Leadership Initiative [grant number 80NSSC22M0070] and the Army Research Office [grant number W911NF2010151].

Minchan Jung was supported by Korea Institute for Advancement of Technology (KIAT) grant funded by the Korea Government (MOTIE), Human Resource Development Program for Industrial Innovation (Global) [grant number RS-2024-00435406].

Appendix

A Index to Multimedia Extensions

Extension	Media type	Description
1	Video	Algorithmic comparison: 2D Car
2	Video	Algorithmic comparison: Quadrotor
3	Video	Algorithmic comparison: Ant
4	Video	Algorithmic comparison: Barkour
5	Video	Hardware experiments for environment 1
6	Video	Hardware experiments for environment 2

B Maximum Entropy DDP

This section provides a derivation of Maximum Entropy DDP (ME-DDP). Starting with the entropic regularized cost and corresponding optimal policy, we derive unimodal and multimodal policies.

B.1 Optimal Policy

To solve (12), we set up a Lagrangian $\mathcal{L}$ with a Lagrangian multiplier $\lambda \in \mathbb{R}$ as:

$$\begin{aligned}\mathcal{L} &= \mathbb{E}_{u \sim \pi} [l(x, u) + \tilde{V}(x')] - \tau \mathcal{H}[\pi(\cdot|x)] \\ &\quad + \lambda \left(1 - \int \pi(u|x) du \right) \\ &= \mathbb{E}_{u \sim \pi} [l(x, u) + \tilde{V}(x') + \tau \ln \pi(u|x) - \lambda] + \lambda \\ &= \int_{\mathbb{R}^{n_u}} \pi(u|x) [\tilde{Q}(x, u) + \tau \ln \pi(u|x) - \lambda] du + \lambda,\end{aligned}$$

where we dropped time instance t and denote x_{t+1} as x' for readability. $\pi(\cdot|x)$ means that it originally was a function of u , but u vanishes after expectation was taken over u . We used the definition of $\tilde{Q}(x, u) = l(x, u) + \tilde{V}(x')$ in the last equality. From the first-order optimality condition, taking the functional derivative of $\mathcal{L}$ with respect to π and setting it zero, we have

$$\tilde{Q}(x, u) + \tau \ln \pi(u|x) - \lambda + \tau = 0.$$

Thus, the optimal policy π^* is obtained as

$$\pi^*(u|x) = \exp \left[-\frac{1}{\tau} (\tilde{Q}(x, u) - \lambda) - 1 \right]. \quad (\text{B.1})$$

From the constraint in (12), we eliminate λ from the above equation. First, we plug (B.1) into the constraint, having

$$\begin{aligned}1 &= \int \pi^*(u|x) du = \int \exp \left[-\frac{1}{\tau} \tilde{Q}(x, u) \right] \exp \left[\frac{\lambda}{\tau} - 1 \right] du \\ &\Leftrightarrow \exp \left[\frac{\lambda}{\tau} - 1 \right] = \left(\int \exp \left[\frac{1}{\tau} \tilde{Q}(x, u) du \right] \right)^{-1} = Z(x)^{-1},\end{aligned} \quad (\text{B.2})$$

where $Z(x)$ is the partition function of π :

$$Z(x) = \int \exp \left[-\frac{1}{\tau} \tilde{Q}(x, u) \right] du. \quad (\text{B.3})$$

By substituting (B.2) into (B.1), the optimal policy is obtained as

$$\begin{aligned}\pi^*(u|x) &= \exp \left[-\frac{1}{\tau} \tilde{Q}(x, u) \right] \exp \left[\frac{\lambda}{\tau} - 1 \right] \\ &= Z(x)^{-1} \exp \left[-\tilde{Q}(x, u)/\tau \right].\end{aligned} \quad (\text{B.4})$$

Value function and Partition function To derive the relationship between the value and partition functions in (13), we transform (B.4) to get

$$\begin{aligned}\tilde{Q}(x, u) &= -\tau \ln(Z(x) \pi^*(u|x)) \\ &= -\tau (\ln Z(x) + \ln \pi^*(u|x)).\end{aligned}$$

In (12), since the entropy $\mathcal{H}[\pi(\cdot|x)]$ is not a function of u , the term can be inside of the expectation:

$$\tilde{V}(x) = \min_{\pi} \{ \mathbb{E}_{u \sim \pi} [l(x, u) + \tilde{V}(x') - \tau \mathcal{H}[\pi(\cdot|x)]] \},$$

From the above two equations, by taking the expectation with respect to the optimal policy, we get the following relationship.

$$\begin{aligned}\tilde{V}(x) &= \mathbb{E}_{\pi^*} [\tilde{Q}(x, u) - \tau \mathcal{H}[\pi^*]] \\ &= \int \pi^*(u|x) [-\tau (\ln Z(x) + \ln \pi^*(u|x)) \\ &\quad + \tau \ln \pi^*(u|x)] du \\ &= -\tau \ln Z(x) \int \pi^*(u|x) du \\ &= -\tau \ln Z(x).\end{aligned} \quad (\text{B.5})$$

Normal and Regularized Value Functions We proceed to establish the relationship between the regularized and normal value functions. Assume that the regularized value function $\tilde{V}_t(x_t)$ is obtained by adding a constant c_t to the normal value function $V_t(x_t)$:

$$\tilde{V}_t(x_t) = V_t(x_t) + c_t \quad (\text{B.6})$$

At the terminal time step $t = T$, the value function is defined by the terminal cost $\Phi(x_T)$ and since the entropy $\mathcal{H}[\pi(\cdot|x_t)]$ is defined only up to $t = T - 1$, we have

$$V_T(x_T) = \tilde{V}_T(x_T) = \Phi(x_T),$$

which implies that the hypothesis holds with $c_T = 0$.

Next, assume the hypothesis (B.6) holds for time step $t + 1$: $\tilde{V}_{t+1}(x_{t+1}) = V_{t+1}(x_{t+1}) + c_{t+1}$. We must show it holds for time step t . From the definition of the regularized

Q function $\tilde{Q}$, we have

$$\begin{aligned}\tilde{Q}_t(x_t, u_t) &= \overbrace{l(x_t, u_t) + V_{t+1}(x_{t+1})}^{\tilde{Q}(x_t, u_t)} + c_{t+1} \\ &= Q(x_t, u_t) + c_{t+1}.\end{aligned}\quad (\text{B.7})$$

To compute integral of $\tilde{Q}$ to obtain the partition function $Z(x)$, we consider a quadratic approximation of Q around a pair of nominal trajectories as in (5). We complete the square of the deviation term with respect to δu :

$$\begin{aligned}Q(x, u) &\approx \underbrace{l(\bar{x}, \bar{u}) + V(\bar{x}')}_{Q(\bar{x}, \bar{u})} + \delta Q(\delta x, \delta u), \\ \delta Q(\delta x, \delta u) &= \frac{1}{2} \delta u^\top Q_{uu} \delta u + [Q_u + Q_{ux} \delta x]^\top \delta u \\ &\quad + \left(Q_x^\top + \frac{1}{2} \delta x^\top Q_{xx} \right) \delta x \\ &= \frac{1}{2} (\delta u - \delta u^*)^\top Q_{uu} (\delta u - \delta u^*) \\ &\quad - \underbrace{\frac{1}{2} \delta u^{*\top} Q_{uu} \delta u^* + \left(Q_x^\top + \frac{1}{2} \delta x^\top Q_{xx} \right) \delta x}_{\delta Q(\delta x, \delta u^*)},\end{aligned}\quad (\text{B.8})$$

where the optimal control δu^* is given in (6). This procedure can be seen as computing the minimizer (δu^*) by completing the square of the quadratic function. Furthermore, since $V_t(x_t) = Q_t(x_t, u_t^*) \approx Q_t(\bar{x}_t, \bar{u}_t) + \delta Q_t(\delta x_t, \delta u_t^*)$, we have

$$Q(x_t, u_t) = V(x_t) + \frac{1}{2} (\delta u - \delta u^*)^\top Q_{uu} (\delta u - \delta u^*),$$

and with (B.7),

$$\tilde{Q}_t(x_t, u_t) = V(x_t) + c_{t+1} + \frac{1}{2} v^\top Q_{uu} v, \quad (\text{B.9})$$

where we put $v = \delta u - \delta u^*$. Assuming that Q_{uu} is PD, the quadratic approximation also allows us to compute Z in (B.3) using Gaussian integral, that is, an integral has a form of

$$\int \exp \left[\frac{1}{2} y^\top A^{-1} y \right] dy = [(2\pi)^{n_y} |A^{-1}|]^{1/2},$$

with $y \in \mathbb{R}^{n_y}$, and a symmetric PD matrix $A \in \mathbb{R}^{n_y \times n_y}$. Substituting (B.9) into (B.3), we have

$$\begin{aligned}Z(x) &= \int \exp \left[-\frac{V(x) + c_{t+1}}{\tau} \right] \exp \left[-\frac{1}{2} v^\top \frac{Q_{uu}}{\tau} v \right] dv \\ &= \exp \left[-\frac{V(x) + c_{t+1}}{\tau} \right] [(2\pi)^{n_u} |\tau Q_{uu}^{-1}|]^{1/2}.\end{aligned}$$

When Q_{uu} is not PD, it needs to be regularized as in (11). We revisit the regularization later when computing the unimodal Gaussian policy. Substituting this Z back into (B.5), the value function is obtained as the sum of value function of normal DDP and a term induced by the Gaussian policy

which we call V_H :

$$\begin{aligned}\tilde{V}(x) &= -\tau \ln Z(x) \\ &= -\tau \left[-\frac{V(x) + c_{t+1}}{\tau} + \frac{1}{2} \ln((2\pi)^{n_u} |\tau Q_{uu}^{-1}|) \right] \\ &= V(x) + c_{t+1} + \underbrace{\frac{\tau}{2} [\ln |Q_{uu}| - n_u \ln(2\pi\tau)]}_{V_H} \\ &= V(x) + \underbrace{c_{t+1} + V_H}_{c_t}.\end{aligned}\quad (\text{B.10})$$

The term c_t is purely a function of c_{t+1} and the Hessian Q_{uu} . Since Q_{uu} is evaluated along the nominal trajectory $(\bar{x}_t, \bar{u}_t)$, and not along (x_t, u_t) , c_t remains state-independent.

From (B.6) and (B.7) and (B.10), the normal value functions (V, Q) and the regularized ones $(\tilde{V}, \tilde{Q})$ are identical up to the constant, therefore, the Ricatti recursion used to propagate derivatives back in time ((8), (9)) are the same among them, and therefore the derivatives are the same, for instance:

$$Q_{uu} = \tilde{Q}_{uu}. \quad (\text{B.11})$$

The difference shows up in zeroth-order terms:

$$\begin{aligned}\tilde{V}_t(\bar{x}_t) &= V_t(\bar{x}_t) + c_t, & \tilde{Q}_t &= Q_t + c_{t+1} \\ c_t &= c_{t+1} + V_{H,t}, & c_T &= 0,\end{aligned}$$

with $t = 1, \dots, T-1$, where c_t captures the effect of the entropy term at time step t .

B.2 Unimodal policy

Plugging (B.7) into the optimal policy in (B.4), and using (B.11), we obtain

$$\begin{aligned}\pi^*(u|x) &\propto \exp \left[-\tilde{Q}(x, u)/\tau \right] \\ &\propto \exp \left[-\frac{1}{2\tau} (\delta u - \delta u^*)^\top Q_{uu} (\delta u - \delta u^*) \right] \\ &\propto \exp \left[-\frac{1}{2} (\delta u - \delta u^*)^\top \Sigma^{-1} (\delta u - \delta u^*) \right],\end{aligned}$$

with $\Sigma = \tau Q_{uu}^{-1}$. This equation indicates that the optimal policy is normally distributed with mean δu^* and covariance τQ_{uu}^{-1} . When Q is not locally convex, we add a regularization term to δQ as in (11). In this case, the covariance is also regularized as

$$\begin{aligned}\delta Q &+ \frac{1}{2} \mu_{\text{reg}} \delta u^\top \delta u \\ &= \frac{1}{2} (\delta u - \delta u^*)^\top Q_{uu}^{\text{reg}} (\delta u - \delta u^*) + (\text{terms without } \delta u),\end{aligned}$$

which yields

$$\pi^*(u|x) \propto \exp \left[-\frac{1}{2\tau} (\delta u - \delta u^*)^\top [Q_{uu}^{\text{reg}}]^{-1} (\delta u - \delta u^*) \right],$$

with δu^* computed with Q_{uu}^{reg} . In fact, the requirement of Q_{uu} being PD serves a dual purpose: it ensures the existence of a unique minimizer for the DDP sub-problem while simultaneously defining a valid covariance for the Gaussian policy. Hereafter, we treat Q_{uu} as a PD matrix. In cases where it is not PD, Q_{uu} implicitly represents the regularized one used to satisfy the requirements of both the DDP backward pass and the covariance.

B.3 Multimodal policy

In this section, we consider approximating the value function using multiple trajectories. We first introduce the additive structure of the value functions. Then, we show how to compose the multimodal policy from the unimodal policies associated with each trajectory.

Additive structure: Instead of performing a single quadratic approximation of the cost around a pair of nominal trajectories, we consider the combined approximation. We start from the terminal cost $\Phi(x_T)$ with N sets of trajectories: $X^{(n)} = [x_1^{(n)}, \dots, x_T^{(n)}]$, $U^{(n)} = [u_1^{(n)}, \dots, u_{T-1}^{(n)}]$, with $n = 1, \dots, N$. Using the LogSumExp operation and the terminal condition (10), we first approximate the value function at terminal timestep T over all N trajectories as

$$\tilde{V}(x_T) = -\tau \ln \sum_{n=1}^N \exp \left[-\frac{1}{\tau} \Phi^{(n)}(x_T) \right], \quad (\text{B.12})$$

where $\Phi^{(n)}(x)$ denotes the terminal cost $\Phi(\cdot)$ evaluated for $x^{(n)}$. To proceed, we introduce the exponential transformation $z_t = \mathcal{E}_\tau[\tilde{V}(x_t)]$, with $\mathcal{E}_\tau(y) = \exp[-y/\tau]$, which transforms the cost and value functions into reward and desirability functions, respectively:

$$r_t = \mathcal{E}_\tau[l_t], \quad r_T = \mathcal{E}_\tau[\Phi(x_T)], \quad z_t = \mathcal{E}_\tau[\tilde{V}(x_t)].$$

From (B.12) and (10), at the terminal time step, z (the approximation by $z^{(n)}$ s) is written as a sum of $z^{(n)}$ s due to the property of the exponential function as below.

$$\begin{aligned} z_T(x_T) &= \exp \left[-\frac{1}{\tau} \left(-\tau \ln \sum_{n=1}^N \exp \left[-\frac{1}{\tau} \Phi^{(n)}(x) \right] \right) \right] \\ &= \sum_{n=1}^N \exp \left[-\frac{1}{\tau} \Phi^{(n)}(x) \right] = \sum_{n=1}^N z_T^{(n)}(x_T). \end{aligned}$$

This additive structure holds not only at the terminal time step, which can be proved by the following induction. Before starting the induction, we establish the following relation:

$$z(x) = \mathcal{E}_\tau[\tilde{V}(x)] = \exp \left[-\frac{1}{\tau} (-\tau \ln Z(x)) \right] = Z(x), \quad (\text{B.13})$$

and

$$\begin{aligned} Z(x) &= \int \exp \left[-\frac{1}{\tau} \underbrace{(l(x, u) + \tilde{V}(x'))}_{\tilde{Q}(x, u)} \right] du \quad (\text{B.14}) \\ &= \int \exp \left[-\frac{1}{\tau} l(x, u) \right] \exp \left[-\frac{1}{\tau} \tilde{V}(x') \right] du \\ &= \int r(x, u) z(x') du, \end{aligned}$$

and therefore,

$$z(x) = \int r(x, u) z(x') du, \quad (\text{B.15})$$

for the global approximation. In addition, the relation also holds for the local trajectories:

$$z^{(n)}(x) = \int r^{(n)}(x, u) z^{(n)}(x') du, \quad (\text{B.16})$$

which can be confirmed by performing (B.13) and (B.14) with the n -th trajectory.

Assume the additive structure holds at time instance $t + 1$:

$$z(x') = \sum_{n=1}^N z^{(n)}(x').$$

Substitution of the equation above into (B.15), at time t , we get

$$\begin{aligned} z(x) &= \int \sum_{n=1}^N z^{(n)}(x') r(x, u) du \\ &= \sum_{n=1}^N \int z^{(n)}(x') r(x, u) du \\ &\approx \sum_{n=1}^N \int z^{(n)}(x') r^{(n)}(x, u) du \quad (\text{B.17}) \\ &= \sum_{n=1}^N z^{(n)}(x). \end{aligned}$$

This implies that the additive structure holds at all time steps by induction.

The approximation in the third line is justified by the local support of the desirability function $z^{(n)}(x)$. When multiplied by $z^{(n)}$, the global reward $r(x, u)$ is effectively filtered, contributing only its local component $r^{(n)}(x, u)$. We note that, if $r(x, u)$ were formally constructed as a sum of local components, this simple additive property would not hold due to cross-terms. This local-filtering property is the key to the derivation.

Computing optimal control and weights: In order to use the additive structure in the optimal control distribution, we first write π^* in terms of r and z as

$$\begin{aligned} \pi^*(u|x) &= Z^{-1} \exp \left[-\frac{1}{\tau} \tilde{Q}(x, u) \right] \\ &= Z^{-1} \exp \left[-\frac{1}{\tau} (l(x, u) + \tilde{V}(x')) \right] \\ &= [z(x)]^{-1} r(x, u) z(x'). \end{aligned}$$

The n -th local policy takes the form of

$$\pi^{*(n)}(u|x) = [z^{(n)}(x)]^{-1} [r^{(n)}(x, u)] z^{(n)}(x').$$

Using the additive structure of z , and from the two equations above, π^* is written as a weighted sum of $\pi^{*(n)}$ s as

$$\begin{aligned} \pi^*(u|x) &= [z(x)]^{-1} \left[\sum_{n=1}^N z^{(n)}(x') \right] r(x, u) \\ &= \sum_{n=1}^N \frac{z^{(n)}(x)}{z(x)} [z^{(n)}(x)]^{-1} z^{(n)}(x') r(x, u) \\ &\approx \sum_{n=1}^N \frac{z^{(n)}(x)}{z(x)} [z^{(n)}(x)]^{-1} z^{(n)}(x') [r^{(n)}(x, u)] \\ &= \sum_{n=1}^N \omega^{(n)}(x) \pi^{*(n)}(u|x), \end{aligned}$$

with the weight

$$\omega^{(n)} = [z(x)]^{-1} z^{(n)}(x), \quad \text{with } \sum_{n=1}^N \omega^{(n)} = 1. \quad (\text{B.18})$$

Each policy for the n th trajectory is given by Gaussian distribution as we saw in the unimodal case:

$$\pi^{(n)}(\delta u^{(n)} | \delta x^{(n)}) = \mathcal{N}(\delta u^{(n)}; \delta u^{*(n)}, \tau(Q_{uu}^{(n)})^{-1}).$$

Since both z and π are a weighted sum of $z^{(n)}$ and $\pi^{(n)}$, the backward pass of ME-DDP is achieved by performing the backward pass in N different nominal trajectories and composing them with weights. To compute the weights, we first represent the desirability function z with the value function $\tilde{V}$. From (B.5) for each trajectories, we have

$$z^{(n)}(x) = \exp \left[-\frac{1}{\tau} \tilde{V}^{(n)}(x) \right]. \quad (\text{B.19})$$

Plugging this $z^{(n)}$ in the right-hand side of (B.17), and Z in (B.5) in the left-hand side of the same equation, we have a similar LogSumExp structure in value function:

$$\tilde{V}(x) = -\tau \ln \sum_{n=1}^N \exp \left[-\frac{1}{\tau} \tilde{V}^{(n)}(x) \right].$$

Plugging the equation above into (B.5) gives

$$z(x) = \exp \left[-\frac{1}{\tau} \tilde{V}(x) \right] = \sum_{n=1}^N \exp \left[-\frac{1}{\tau} \tilde{V}^{(n)}(x) \right].$$

From the above equation, (B.19), and (B.18), we get

$$\omega_t^{(n)} = z_t(x_t)^{-1} z_t^{(n)}(x_t) = \frac{\exp \left[-\frac{1}{\tau} (\tilde{V}_t^{(n)}(x_t)) \right]}{\sum_{n=1}^N \exp \left[-\frac{1}{\tau} \tilde{V}_t^{(n)}(x_t) \right]},$$

where we explicitly write V as a function of state and time by recovering the time index in the notation. The value function $\tilde{V}_t(x_t)$ can be approximated by the following quadratic approximation and recursion:

$$\begin{aligned} \tilde{V}_t^{(n)}(x) &\approx V_t^{(n)}(\bar{x}^{(n)}) + c_t + V_{x,t}^{(n)}(\bar{x}^{(n)})^\top \delta x^{(n)} \\ &\quad + \frac{1}{2} \delta x^{(n)\top} V_{xx,t}^{(n)}(\bar{x}^{(n)}) \delta x^{(n)}, \\ c_t &= c_{t+1} + V_{H,t}^{(n)}, \end{aligned}$$

with $c_T = 0$, $t = 1, \dots, T-1$. that uses derivatives from the backward pass of normal deterministic DDP and δx from the forward pass. Although the policy is defined at every time step, independent sampling at each time step t can lead to physically inconsistent control jitter, due to the mode switching. To ensure temporal coherence, we therefore pick the mode to sample based on the weight at time $t = 1$ (w_1) and keep sampling from the same mode throughout the time horizon.

B.4 Pseudocode

We provide the exploration mechanisms for UG-ME-DDP and MG-ME-DDP in Algorithms B.1 and B.2, respectively. These exploration mechanisms can be used in the main

algorithm in Algorithm 1. We note that in the formulation proposed in So et al. (2022), the number of trajectories was fixed ($N = 2$) in UG-ME-DDP. To isolate the sampling density and the property of the policy, here we allow $N \geq 2$.

In UG-ME-DDP, the policy is composed base on the best trajectory. In MG-ME-DDP, we need $V_H = \{V_{H,1:T-1}\}_{n=1}^N$ that accounts for the effect of entropy to compose the multimodal policy.

Algorithm B.1: Exploration by Unimodal Gaussian Policy

Input: U, X, Q_{uu}, K, b
Output: Updated trajectories X, U
1 **for** $n = 1$ **to** $N, n \neq b$ **in parallel do**
2 $X^{(n)}, U^{(n)} \leftarrow$
 SampleFromUG($X^{(b)}, U^{(b)}, Q_{uu}^{(b)}, K^{(b)}$)
3 **end**
4 **return** $\{X, U\}$

Algorithm B.2: Exploration by Multimodal Gaussian Policy

Input: U, X, Q_{uu}, K, V_H, b
Output: Updated trajectories X, U
1 **for** $n = 1$ **to** $N, n \neq b$ **in parallel do**
2 $X^{(n)}, U^{(n)} \leftarrow$
 SampleFromMG(X, U, Q_{uu}, K, V_H)
3 **end**
4 **return** $\{X, U\}$

C Positive Definiteness of Control Hessian

This section is to provide additional explanation for Geometric Analysis and Algorithmic Synthesis section. In this section, we show how $Q_{uu,t}$ can be PD given the following conditions:

1. First-order dynamics: The transition dynamics are locally linear, $\delta x_{t+1} = f_x \delta x_t + f_u \delta u_t$. meaning second-order derivatives of the dynamics (f_{xx}, f_{uu}, f_{xu}) are zero.
2. PD Terminal and running cost: The Hessian of the terminal cost and therefore that of the value function is PD $V_{xx,T} = \Phi_{xx,T} \succ 0$. The Hessians of the running cost are PD $l_{uu,t}, l_{xx,t} \succ 0$ for $t = 1, \dots, T-1$.
3. Running cost is decoupled: $l_{xu} = 0_{n_x, n_u}$.

We note that this is not a restrictive assumption. Standard design choices, such as quadratic cost or PD Hessian approximation, satisfy these requirements. We note that this is not a restrictive assumption. Standard design choices, such as quadratic or cost with PD Hessian approximation, can satisfy these requirements.

With conditions above, we can show that at time step $T-1$, $Q_{uu,T-1}$ is PD:

$$Q_{uu,T-1} = \underbrace{l_{uu,T-1}}_{\succ 0} + \underbrace{f_{u,T-1}^\top V_{xx,T} f_{u,T-1}}_{\succeq 0} \succ 0, \quad (\text{C.1})$$

where the term $f_u^\top V_{xx,T} f_u$ is at least positive semi-definite (PSD) because $V_{xx,T} \succ 0$.

Assume the Hessian of the value function at time step $t+1$ satisfies $V_{xx,t+1} \succ 0$. We now show by induction that $Q_{uu,t}$ is PD and $V_{xx,t}$ remains PD for all t . By this assumption and the condition $l_{uu,t} \succ 0$, the matrix $Q_{uu,t}$ is PD which follows same argument as in (C.1).

The Riccati recursion for the value function Hessian is:

$$V_{xx,t} = Q_{xx,t} - Q_{xu,t} Q_{uu,t}^{-1} Q_{ux,t},$$

which is obtained by substituting into (9) where Q_{uu} terms in gains cancel out.

A symmetric block matrix M is positive definite if and only if its blocks satisfy certain conditions:

$$\begin{aligned} M = \begin{bmatrix} Q_{xx} & Q_{xu} \\ Q_{ux} & Q_{uu} \end{bmatrix} \succ 0 \\ \iff Q_{uu} \succ 0 \text{ and } Q_{xx} - Q_{ux}^\top Q_{uu}^{-1} Q_{ux} \succ 0. \end{aligned}$$

The term $V_{xx} = Q_{xx} - Q_{ux}^\top Q_{uu}^{-1} Q_{ux}$ is the Schur Complement of Q_{uu} in the matrix M . Therefore, to prove $V_{xx,t} \succ 0$, it is enough to show that joint Hessian of the Q -function M is PD.

By constructions in (8), we can decompose M into:

$$M = \begin{bmatrix} l_{xx} & l_{xu} \\ l_{ux} & l_{uu} \end{bmatrix} + \begin{bmatrix} f_x \\ f_u \end{bmatrix}^\top V_{xx,t+1} \begin{bmatrix} f_x & f_u \end{bmatrix} \succ 0.$$

From the assumption that $l_{xu} = O_{n_x, n_u}$, M is a sum of PD and PSD matrices and therefore $M \succ 0$. Therefore, it follows by backward induction over time that $Q_{uu,t}$ and $V_{xx,t}$ are PD for all time steps.

D Block-diagonal approximation in SVNM

In this section, we examine the intuition of the approximation employed in SVNM. We first consider the size of the Hessian. Here, We follow the same notation as in Section [Stein Variational Dynamic Optimization](#).

Suppose that we concatenate the particles to get

$$X = [x_1^\top, x_2^\top, \dots, x_N^\top]^\top \in \mathbb{R}^{Nd}.$$

Then, the full Hessian of the KL function is

$$H = \nabla_X^2 D_{\text{KL}}(q||p) \in \mathbb{R}^{Nd \times Nd}.$$

In a block view, it can be written as

$$H = \begin{bmatrix} H^{1,1} & H^{1,2} & \dots & H^{1,N} \\ H^{2,1} & H^{2,2} & \dots & H^{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ H^{N,1} & H^{N,2} & \dots & H^{N,N} \end{bmatrix},$$

with

$$H^{s,n} = \frac{\partial^2 D_{\text{KL}}}{\partial x_s \partial x_n} \in \mathbb{R}^{d \times d}$$

and $s, n = 1 \dots N$. Here, each block $H^{s,n}$ encodes how the update of particle s depends on the particle n . The diagonal blocks are the local curvature of the KL functional associated

with particle x_s , ignoring interactions with other particles. There are N^2 blocks each of which has d^2 entries.

In expressions such as (19), the term $h_{ij}(x_s, x_n)$ is i, j element of block $H^{s,l}$:

$$H_{i,j}^{s,n} = h_{ij}(x_s, x_n), \quad i, j = 1, \dots, d.$$

To understand the effect of the block-diagonal approximation intuitively, we consider a simple case with $N = 2$. Then, the equation for the Newton direction is:

$$\begin{bmatrix} H^{1,1} & H^{1,2} \\ H^{2,1} & H^{2,2} \end{bmatrix} \begin{bmatrix} \beta^1 \\ \beta^2 \end{bmatrix} = - \begin{bmatrix} \nabla \hat{J}[\mathbf{0}](x_1) \\ \nabla \hat{J}[\mathbf{0}](x_2) \end{bmatrix}.$$

Now, by focusing on the resulting two equations, we can see that they can be written as

$$\sum_{n=1}^N H^{s,n} \beta^n = -\nabla \hat{J}[\mathbf{0}](x_s), \quad s = 1, \dots, N.$$

The update direction β^s of particle s depends on the update of all other particles β^n . The only way to solve the system of equations is to invert the large H matrix, which is expensive. An alternative performed in the approximation is to keep the terms only with $n = s$ in the summation, which yields

$$H^{s,s} \beta^s = -\nabla \hat{J}[\mathbf{0}](x_s), \quad s = 1, \dots, N,$$

in the previous simple example with $N = 2$, the system becomes

$$\begin{bmatrix} H^{1,1} & O_d \\ O_d & H^{2,2} \end{bmatrix} \begin{bmatrix} \beta^1 \\ \beta^2 \end{bmatrix} = - \begin{bmatrix} \nabla \hat{J}[\mathbf{0}](x_1) \\ \nabla \hat{J}[\mathbf{0}](x_2) \end{bmatrix}$$

Under this approximation, instead of inverting $H \in \mathbb{R}^{Nd \times Nd}$, we only need to invert the N diagonal blocks $H^{s,s} \in \mathbb{R}^{d \times d}$ independently, allowing the β^s to be computed in parallel. With these, we recover the direction $w(x_s)$ by

$$w(x_s) = \sum_{n=1}^N \beta^n k(x_n, x_s),$$

which coincides with (20).

E DDP and Barrier Function

This section provides how the relaxed-log barrier function modifies DDP. Here, we use the concatenated state and control as $y_t = [x_t^\top, u_t^\top]^\top$. Since the barrier term is a part of cost, it modifies the Q function of the standard DDP, yielding $\hat{Q}$, whose derivatives are given as:

$$\hat{Q}_y = Q_y + \nabla_y \mathcal{B}_r(g(y)), \quad \hat{Q}_{yy} = Q_{yy} + \nabla_y^2 \mathcal{B}_r(g(y)).$$

The Hessian of the barrier function is given as follows:

$$\begin{aligned} & \nabla_y^2 \mathcal{B}_r(g_i; \mu) \\ &= \begin{cases} \frac{\mu}{g_i^2} \nabla g_i (\nabla g_i)^\top - \frac{\mu}{g_i} \nabla^2 g_i, & g_i \leq -\delta_\mu, \\ \frac{\mu}{\delta_\mu^2} \nabla g_i (\nabla g_i)^\top + \frac{\mu(g_i + 2\delta_\mu)}{\delta_\mu^2} \nabla^2 g_i, & g_i > -\delta_\mu. \end{cases} \end{aligned}$$

To keep the Hessian well-conditioned, we drop the second-order terms ($\nabla^2 g_i$), which can be negative definite.

F Numerical Sensitivity of Relaxed-Barrier Function

This section provides a detailed analysis of the numerical limitations of the relaxed-barrier formulation introduced in Section **Geometric Analysis and Algorithmic Synthesis**. We identify two primary sources of sensitivity: the inherent scaling of the log barrier gradient and the hard branching mechanism introduced by the relaxation.

Scaling of Gradient The numerical sensitivity arises from the inherent scaling of the gradient near the boundary of the constraints $g(x) \approx 0$ in the exact log barrier and a small switching point δ_μ in the relaxed one. Consider the gradient of the relaxed barrier function $\mathcal{B}_r(x)$ provided in (25):

$$\nabla \mathcal{B}_r(x) = \begin{cases} -\frac{\nabla g(x)}{g(x)}, & g(x) \leq -\delta_\mu, \\ \left(\frac{g(x) + 2\delta_\mu}{\delta_\mu^2} \right) \nabla g(x), & g(x) > -\delta_\mu. \end{cases}$$

This scaling amplifies small numerical discrepancies. Suppose that we execute the algorithm across different computational environments (e.g., CPU vs. GPU or different hardware architectures). We denote values computed on different devices with superscripts a and b . Small differences in $\nabla g(x)$, from device-specific optimizations or floating-point rounding errors, are amplified. For two devices with slightly different gradients $\nabla g^a(x)$ and $\nabla g^b(x) = \nabla g^a(x) + \epsilon$, the discrepancy in the gradient around the boundary $g(x) \approx -\delta_\mu$ satisfies:

$$\|\nabla \mathcal{B}_r^a(x) - \nabla \mathcal{B}_r^b(x)\| \approx \begin{cases} \frac{\|\nabla g^a(x) - \nabla g^b(x)\|}{|g(x)|} = \frac{\epsilon}{|g(x)|} \approx \frac{\epsilon}{\delta_\mu}, & g(x) \leq -\delta_\mu, \\ \frac{g(x) + 2\delta_\mu}{\delta_\mu^2} \|\nabla g_1(x) - \nabla g_2(x)\| \approx \frac{\epsilon}{\delta_\mu}, & g(x) > -\delta_\mu. \end{cases}$$

For example, if $\epsilon = 10^{-11}$ and the relaxation parameter is small (e.g., $\delta_\mu = 10^{-6}$), the discrepancy is amplified to the order of 10^{-5} . These errors accumulate through the forward and backward passes of the DDP, ultimately showing up as difference in trajectories. While a small δ_μ is often necessary for strict constraint satisfaction, especially in MPC, it introduces a source of device-dependent sensitivity. We note that this is not unique to the relaxed formulation. The exact barrier also has the same issue near the boundary of the constraints.

Hard Branching (Unique to the relaxed formulation.) Relaxed log-barrier functions switch from the standard log to a quadratic approximation at $g(x) = -\delta_\mu$. This introduces a hard branch, that is, the code must choose which branch to evaluate for each state. Even if the formulation is $\mathcal{C}^2$ continuous at the switching point, a small floating-point difference from, for example, order of operations, can make one evaluation select the log branch while another selects the quadratic branch. This results in slightly different derivatives that propagate and accumulate through DDP, as before. The effect is more pronounced for very small δ_μ , where even small deviations can create a large difference around δ_μ .

References

- Aguiar AP, Bayer FA, Hauser J, Häusler AJ, Notarstefano G, Pascoal AM, Rucco A and Saccon A (2017) *Constrained Optimal Motion Planning for Autonomous Vehicles Using PRONTO*. Cham: Springer International Publishing. ISBN 978-3-319-55372-6, pp. 207–226. DOI:10.1007/978-3-319-55372-6_10.
- Aoyama Y, Lehmann P and Theodorou EA (2025) Second-order stein variational dynamic optimization. In: *2025 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 7209–7216. DOI:10.1109/ICRA55743.2025.11128381.
- Aoyama Y and Theodorou EA (2024) Generalized maximum entropy differential dynamic programming. In: *2024 IEEE 63rd Conference on Decision and Control (CDC)*. pp. 8825–8831. DOI:10.1109/CDC56724.2024.10886519.
- Barcelos L, Lai T, Oliveira R, Borges P and Ramos F (2024) Path signatures for diversity in probabilistic trajectory optimisation. *The International Journal of Robotics Research* 43(11): 1693–1710. DOI:10.1177/02783649241233300.
- Betts JT (2001) *Practical Methods for Optimal Control Using Nonlinear Programming*. Philadelphia, PA: SIAM.
- Boutselis GI and Theodorou E (2021) Discrete-time differential dynamic programming on lie groups: Derivation, convergence analysis, and numerical results. *IEEE Transactions on Automatic Control* 66(10): 4636–4651. DOI:10.1109/TAC.2020.3034206.
- Bradbury J, Frostig R, Hawkins P, Johnson MJ, Katariya Y, Leary C, Maclaurin D, Necula G, Paszke A, VanderPlas J, Wanderman-Milne S and Zhang Q (2018) JAX: composable transformations of Python+NumPy programs.
- Caluwaerts K, Iscen A, Kew JC, Yu W, Zhang T, Freeman D, Lee KH, Lee L, Saliceti S, Zhuang V, Batchelor N, Bohez S, Casarini F, Chen JE, Cortes O, Coumans E, Dostmohamed A, Dulac-Arnold G, Escontrela A, Frey E, Hafner R, Jain D, Jyenis B, Kuang Y, Lee E, Luu L, Nachum O, Oslund K, Powell J, Reyes D, Romano F, Sadeghi F, Sloat R, Tabanpour B, Zheng D, Neunert M, Hadsell R, Heess N, Nori F, Seto J, Parada C, Sindhvani V, Vanhoucke V and Tan J (2023) Barkour: Benchmarking animal-level agility with quadruped robots.
- Carvalho J, Le AT, Baierl M, Koert D and Peters J (2023) Motion planning diffusion: Learning and planning of robot motions with diffusion models. In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 1916–1923. DOI:10.1109/IROS55552.2023.10342382.
- Chen P and Ghattas O (2020) Projected stein variational gradient descent. In: Larochelle H, Ranzato M, Hadsell R, Balcan M and Lin H (eds.) *Advances in Neural Information Processing Systems*, volume 33. Curran Associates, Inc., pp. 1947–1958.
- Chen P, Wu K, Chen J, O’Leary-Roseberry T and Ghattas O (2019) Projected stein variational newton: A fast and scalable bayesian inference method in high dimensions. In: Wallach H, Larochelle H, Beygelzimer A, d’Alché-Buc F, Fox E and Garnett R (eds.) *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.
- Cho GM (2005) Log-barrier method for two-stage quadratic stochastic programming. *Applied Mathematics and Computation* 164(1): 45–69. DOI:https://doi.org/10.1016/j.amc.2004.04.095.
- Deits R and Tedrake R (2015) Computing large convex regions of obstacle-free space for robotic navigation. *The International*

- Journal of Robotics Research* 34(7): 955–977.
- Detommaso G, Cui T, Marzouk Y, Spantini A and Scheichl R (2018) A stein variational newton method. In: Bengio S, Wallach H, Larochelle H, Grauman K, Cesa-Bianchi N and Garnett R (eds.) *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.
- Dong J and Tong XT (2021) Replica exchange for non-convex optimization. *J. Mach. Learn. Res.* 22(1).
- Feller C and Ebenbauer C (2017a) Relaxed logarithmic barrier function based model predictive control of linear systems. *IEEE Transactions on Automatic Control* 62(3): 1223–1238. DOI:10.1109/TAC.2016.2582040.
- Feller C and Ebenbauer C (2017b) A stabilizing iteration scheme for model predictive control based on relaxed barrier functions. *Automatica* 80: 328–339. DOI:https://doi.org/10.1016/j.automatica.2017.02.001.
- Fiacco AV and McCormick GP (1968) *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*. John Wiley and Sons, New York.
- Freeman CD, Frey E, Raichuk A, Girgin S, Mordatch I and Bachem O (2021) Brax - a differentiable physics engine for large scale rigid body simulation.
- Gandhi MS, Vlahov B, Gibson J, Williams G and Theodorou EA (2021) Robust model predictive path integral control: Analysis and performance guarantees. *IEEE Robotics and Automation Letters* 6(2): 1423–1430. DOI:10.1109/LRA.2021.3057563.
- Garreau D, Jitkrittum W and Kanagawa M (2018) Large sample analysis of the median heuristic. *arXiv preprint arXiv:1707.07269*.
- Gill PE, Jay LO, Leonard MW, Petzold LR and Sharma V (2000) An sqp method for the optimal control of large-scale dynamical systems. *Journal of Computational and Applied Mathematics* 120(1): 197–213. DOI:https://doi.org/10.1016/S0377-0427(00)00310-1.
- Gill PE, Murray W and Saunders MA (2002) Snopt: An sqp algorithm for large-scale constrained optimization. *SIAM Journal on Optimization* 12(4): 979–1006. DOI:10.1137/S1052623499350013.
- Gorham J and Mackey L (2017) Measuring sample quality with kernels. In: *Proceedings of the 34th International Conference on Machine Learning*, volume 70. pp. 1292–1301.
- Grandia R, Farshidian F, Ranftl R and Hutter M (2019) Feedback mpc for torque-controlled legged robots. In: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE Press, p. 4730–4737. DOI:10.1109/IROS40897.2019.8968251.
- Harford T (2016) *Messy: How to Be Creative and Resilient in a Tidy-Minded World*. Little, Brown Book Group. ISBN 9781408706770.
- Hauser J (2002) A projection operator approach to the optimization of trajectory functionals. *IFAC Proceedings Volumes* 35(1): 377–382. DOI:https://doi.org/10.3182/20020721-6-ES-1901.00312. 15th IFAC World Congress.
- Hauser J and Saccon A (2006) A barrier function method for the optimization of trajectory functionals with constraints. In: *Proceedings of the 45th IEEE Conference on Decision and Control*. pp. 864–869. DOI:10.1109/CDC.2006.377331.
- Hestenes MR (1969) Multiplier and gradient methods. *Journal of Optimization Theory and Applications* 4: 303–320. DOI:10.1007/BF00927673.
- Huang H, Sundaralingam B, Mousavian A, Murali A, Goldberg K and Fox D (2024) Diffusionseeder: Seeding motion optimization with diffusion for rapid motion planning. In: *Proceedings of the 8th Annual Conference on Robot Learning (CoRL)*. Munich, Germany.
- Jacobson DH and Mayne DQ (1970) *Differential dynamic programming*. Elsevier. ISBN 0-444-00070-4.
- Jr AEB and Ho YC (1975) *Applied Optimal Control: Optimization, Estimation, and Control*. Washington, DC: Hemisphere Publishing Corporation.
- Kalakrishnan M, Chitta S, Theodorou E, Pastor P and Schaal S (2011) Stomp: Stochastic trajectory optimization for motion planning. In: *2011 IEEE international conference on robotics and automation*. IEEE, pp. 4569–4574.
- Kaufman H (1964) The mathematical theory of optimal processes, by I. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko. authorized translation from the Russian. translator: K. N. Trigonoff, editor: L. W. Neustadt. Interscience Publishers (division of John Wiley and Sons, Inc., New York) 1962. viii + 360 pages. *Canadian Mathematical Bulletin* 7(3): 500–500. DOI:10.1017/S0008439500032112.
- Kim MG, Jung M, Hong J and Kim KKK (2025) Mppi-ddp: A hybrid method of collision-free smooth trajectory generation for autonomous robots. *IEEE Transactions on Industrial Informatics* 21(7): 5037–5046. DOI:10.1109/TII.2024.3507940.
- Kuindersma S, Deits RF, Fallon M, Valenzuela A, Dai H, Permenter F, Cooley T and Tedrake R (2016) Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot. *Autonomous Robots* 40(3): 429–455.
- Lambert A and Boots B (2021) Entropy regularized motion planning via stein variational inference.
- Lambert A, Ramos F, Boots B, Fox D and Fishman A (2021) Stein variational model predictive control. In: Kober J, Ramos F and Tomlin C (eds.) *Proceedings of the 2020 Conference on Robot Learning, Proceedings of Machine Learning Research*, volume 155. PMLR, pp. 1278–1297.
- Le AT, Chalvatzaki G, Biess A and Peters J (2023) Accelerating motion planning via optimal transport. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Lee D, Ramos F, Lerch C and Abraham I (2024) Stein variational ergodic search. In: *Proceedings of Robotics: Science and Systems (RSS)*. Delft, Netherlands.
- Lee Y, Li AZ, Huang P, Heiden E, Jatavallabhula KM, Damken F, Smith K, Nowrouzezahrai D, Ramos F and Shkurti F (2025) Stamp: Differentiable task and motion planning via stein variational gradient descent. *IEEE Robotics and Automation Letters* 10(6): 6007–6014. DOI:10.1109/LRA.2025.3561575.
- Levine S (2018) Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*.
- Levine S and Koltun V (2013) Guided policy search. In: Dasgupta S and McAllester D (eds.) *Proceedings of the 30th International Conference on Machine Learning, Proceedings of Machine Learning Research*, volume 28. Atlanta, Georgia, USA: PMLR, pp. 1–9.
- Li W and Todorov E (2004) Iterative linear quadratic regulator design for nonlinear biological movement systems. In: *Proceedings of the 1st International Conference on Informatics in Control, Automation and Robotics (ICINCO)*, volume 1.

- Setúbal, Portugal, pp. 222–229.
- Liao LZ (1996) Global convergence of differential dynamic programming and newton's method for discrete-time optimal control. Technical report, Department of Mathematics, Hong Kong Baptist University, Kowloon, Hong Kong.
- Liao LZ and Shoemaker C (1991) Convergence in unconstrained discrete-time differential dynamic programming. *IEEE Transactions on Automatic Control* 36(6): 692–706. DOI: 10.1109/9.86943.
- Liu Q (2017) Stein variational gradient descent as gradient flow. In: Guyon I, Luxburg UV, Bengio S, Wallach H, Fergus R, Vishwanathan S and Garnett R (eds.) *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Liu Q and Wang D (2016) Stein variational gradient descent: A general purpose variational inference algorithm. In: *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29.
- Liu Y, Ramachandran P, Liu Q and Peng J (2017) Stein variational policy gradient. In: Elidan G, Kersting K and Ihler A (eds.) *Proceedings of the Thirty-Third Conference on Uncertainty in Artificial Intelligence, UAI 2017, Sydney, Australia, August 11–15, 2017*. AUAI Press.
- Luukkonen T (2011) Modelling and control of quadcopter. Technical report, Aalto University, Independent Research Project in Applied Mathematics, Espoo, Finland.
- Meier L, Honegger D and Pollefeys M (2015) Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In: *2015 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 6235–6240. DOI: 10.1109/ICRA.2015.7140074.
- Microsoft (2021) Neural Network Intelligence.
- Miura T, Akai N, Honda K and Hara S (2024) Spline-interpolated model predictive path integral control with stein variational inference for reactive navigation. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 13171–13177. DOI:10.1109/ICRA57147.2024.10610501.
- Murray W and Wright MH (1994) Line search procedures for the logarithmic barrier function. *SIAM Journal on Optimization* 4(2): 229–246. DOI:10.1137/0804013.
- Nocedal J and Wright S (2006) *Numerical optimization*. Springer Science & Business Media.
- Osa T (2020) Multimodal trajectory optimization for motion planning. *The International Journal of Robotics Research* 39: 027836492091829. DOI:10.1177/0278364920918296.
- O'Neill M and Wright SJ (2020) A log-barrier newton-cg method for bound constrained optimization with complexity guarantees. *IMA Journal of Numerical Analysis* 41(1): 84–121. DOI:10.1093/imanum/drz074.
- Pan C, Yi Z, Shi G and Qu G (2024) Model-based diffusion for trajectory optimization. In: *Advances in Neural Information Processing Systems*, volume 37.
- Powell MJD (1969) A method for nonlinear constraints in minimization problems. In: Fletcher R (ed.) *Optimization*. New York, NY: Academic Press, pp. 283–298.
- Power T and Berenson D (2024) Constrained stein variational trajectory optimization. *IEEE Transactions on Robotics* 40: 3602–3619. DOI:10.1109/TRO.2024.3428428.
- Puterman ML (1994) *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. New York, NY, USA: John Wiley & Sons, Inc. ISBN 978-0471619772.
- Rao C, Wright S and Rawlings J (1998) Application of interior-point methods to model predictive control. *Journal of Optimization Theory and Applications* 99(3): 723–757. DOI: 10.1023/A:1021711402723.
- Rockafellar RT (1973) The multiplier method of Hestenes and Powell applied to convex programming. *Journal of Optimization Theory and Applications* 12(6): 555–562.
- Schulman J, Duan Y, Ho J, Lee AX, Awwal I, Bradlow H, Pan J, Patil S, Goldberg K and Abbeel P (2014) Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research* 33(9): 1251–1270. DOI:10.1177/0278364914528132.
- Sleiman JP, Farshidian F and Hutter M (2021) Constraint handling in continuous-time DDP-based model predictive control. In: *2021 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 8209–8215. DOI:10.1109/ICRA48506.2021.9560795.
- So O, Wang Z and Theodorou EA (2022) Maximum entropy differential dynamic programming. In: *2022 International Conference on Robotics and Automation (ICRA)*. pp. 3422–3428. DOI:10.1109/ICRA46639.2022.9812228.
- Sundaralingam B, Hari SKS, Fishman A, Garrett C, Van Wyk K, Blukis V, Millane A, Oleynikova H, Handa A, Ramos F, Ratliff N and Fox D (2023) Curobo: Parallelized collision-free robot motion generation. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 8112–8119. DOI: 10.1109/ICRA48891.2023.10160765.
- Tassa Y, Erez T and Todorov E (2012) Synthesis and stabilization of complex behaviors through online trajectory optimization. In: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, pp. 4906–4913. DOI:10.1109/IROS.2012.6386025.
- Tassa Y, Mansard N and Todorov E (2014) Control-limited differential dynamic programming. In: *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 1168–1175. DOI:10.1109/ICRA.2014.6907001.
- Trivedi S, Wang J, Kpotufe S and Shakhnarovich G (2014) A consistent estimator of the expected gradient outerproduct. In: *Proceedings of the Thirtieth Conference on Uncertainty in Artificial Intelligence, UAI'14*. Arlington, Virginia, USA: AUAI Press. ISBN 9780974903910, p. 819–828.
- Urain J, Le AT, Lambert A, Chaltatzaki G, Boots B and Peters J (2022) Learning implicit priors for motion optimization. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 7672–7679. DOI:10.1109/IROS47612.2022.9981264.
- Vlahov B, Gibson J, Fan DD, Spieler P, Agha-mohammadi Aa and Theodorou EA (2024) Low frequency sampling in model predictive path integral control. *IEEE Robotics and Automation Letters* 9(5): 4543–4550. DOI:10.1109/LRA.2024.3382530.
- Wächter A and Biegler LT (2006) On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming* 106(1): 25–57. DOI:10.1007/s10107-004-0559-y.
- Wang Z, So O, Gibson J, Vlahov B, Gandhi MS, Liu GH and Theodorou EA (2021) Variational inference mpc using tsallis divergence. In: *Robotics: Science and Systems (RSS)*. p. 73. Held virtually, July 12–16, 2021.

- Williams G, Aldrich A and Theodorou EA (2017) Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics* 40(2): 344–357.
- Williams G, Drews P, Goldfain B, Rehg JM and Theodorou EA (2016) Aggressive driving with model predictive path integral control. In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 1433–1440. DOI:10.1109/ICRA.2016.7487277.
- Williams G, Drews P, Goldfain B, Rehg JM and Theodorou EA (2018a) Information-theoretic model predictive control: Theory and applications to autonomous driving. *IEEE Transactions on Robotics* 34(6): 1603–1622. DOI:10.1109/TRO.2018.2865891.
- Williams G, Goldfain B, Drews P, Saigol K, Rehg J and Theodorou E (2018b) Robust sampling based model predictive control with sparse objective information. In: *Proceedings of Robotics: Science and Systems*. Pittsburgh, Pennsylvania. DOI:10.15607/RSS.2018.XIV.042.
- Wills AG and Heath WP (2004) Barrier function based model predictive control. *Automatica* 40(8): 1415–1422. DOI:https://doi.org/10.1016/j.automatica.2004.03.002.
- Wilson RB (1963) *A simplicial algorithm for concave programming*. PhD Thesis, Graduate School of Business Administration, Harvard University, Cambridge, MA, USA.
- Xue H, Pan C, Yi Z, Qu G and Shi G (2025) Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing. In: *2025 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 4974–4981. DOI: 10.1109/ICRA55743.2025.11127320.
- Yi Z, Pan C, He G, Qu G and Shi G (2024) CoVO-MPC: Theoretical analysis of sampling-based mpc and optimal covariance design. *arXiv preprint arXiv:2401.07369*.
- Yin Z, Lai T, Barcelos L, Jacob J, Li Y and Ramos F (2025) Diverse motion planning with stein diffusion trajectory inference. In: *2025 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 15610–15616. DOI:10.1109/ICRA55743.2025.11127953.
- Zhang B, Zhang Y, Zhu H, Yan S, Brox T and Boedecker J (2025) Constrained reinforcement learning with smoothed log barrier function. *Transactions on Machine Learning Research*.
- Zhuo J, Liu C, Shi J, Zhu J, Chen N and Zhang B (2018) Message passing stein variational gradient descent. In: Dy J and Krause A (eds.) *Proceedings of the 35th International Conference on Machine Learning, Proceedings of Machine Learning Research*, volume 80. PMLR, pp. 6018–6027.
- Ziebart BD (2010) *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD Thesis, Carnegie Mellon University, Pittsburgh, PA, USA. CMU-CS-10-110.
- Zucker M, Ratliff N, Dragan AD, Pivtoraiko M, Klingensmith M, Dellin CM, Bagnell JA and Srinivasa SS (2013) Chomp: Covariant hamiltonian optimization for motion planning. *The International Journal of Robotics Research* 32(9-10): 1164–1193. DOI:10.1177/0278364913488805.